VisionGuard: Production Engineering Report for AI vs. Real Image Classification and Containerized Cloud Deployment

Student Name: Shoaib Asif | Roll Number: 23L0710 | Course: Applied Machine Learning Operations (MLOps) Project Repository & Service Prefix: 23L0710_A2_Ops

Executive Summary & Project Provenance

VisionGuard is an end-to-end production machine learning system engineered to detect synthetic and AI-generated imagery from authentic digital photographs. The project demonstrates a rigorous MLOps lifecycle: systematic data auditing and versioning via DVC, reproducible model experimentation with TensorFlow/Keras and MLflow, centralized model governance via DagsHub Model Registry, resilient web serving through Flask, persistent request auditing via PostgreSQL/SQLite, immutable multi-service containerization via Docker Compose, automated CI validation via GitHub Actions, and scalable public cloud deployment on Render. The complete system is publicly reachable and fully reproducible across external networks.

Project Deliverables & Repository Links

Student Name: Shoaib Asif

Student Roll Number: 23L0710 (Standard naming convention: 23L0710_A2_Ops applied everywhere)

GitHub Repository: https://github.com/shoaib-asiff/23L0710_A2_Ops

DagsHub MLOps Project: https://dagshub.com/shoaib-asiff/23L0710_A2_Ops

Live Cloud API URL: https://visionguard-api-23l0710.onrender.com

Graded Commit Hash: 3fdec2ca8358d289013f38474b5b2af8494b69b6

Champion Model Registry Alias: models:/VisionGuard_Classifier@champion (Version 2 on DagsHub)

Disclosure of Technical Documentation & AI Usage

In accordance with institutional guidelines on academic integrity, external resources utilized during this project were strictly limited to official software documentation, framework reference manuals, and standardized architectural tutorials: (1) TensorFlow and Keras 2.15 official documentation for data pipeline design and transfer learning APIs; (2) MLflow 2.x and DagsHub documentation for remote tracking server integration and model registry client methods; (3) Docker and Docker Compose documentation for multi-stage image optimization, non-root user configuration, and bridge network security; (4) Flask 3.0 documentation for streaming request validation and secure file upload handling; and (5) Render.com deployment documentation for Gunicorn worker configuration under constrained memory tiers. All pipeline code, model architectures, database schemas, test harnesses, container definitions, and report commentary were developed and debugged independently.

Chapter 1: Task 1: Data Understanding and Versioning (7.0 Marks)

1.1 Environment Setup and Hardware Verification (1.0 Mark)

What Was Done

An isolated Python virtual environment (.venv) was provisioned using Python 3.10.19. Core scientific and production dependencies were pinned and installed: TensorFlow 2.15.0, Keras 3.x, DVC 3.64.0, MLflow 2.22.2, Flask 3.1.3, SQLAlchemy 2.0.46, Gunicorn 21.2.0, NumPy, Pandas, Scikit-learn, and Pillow. A

hardware and dependency audit script (src/verify_env.py) was executed to log exact component versions and query hardware acceleration capabilities.

Why It Was Done

Strict virtual environment isolation is mandatory in production machine learning systems to prevent package drift and environment poisoning. Recording exact package versions enables 100% deterministic reproducibility across development workstations, continuous integration runners, and production cloud containers.

How It Works in My Own Words

The virtual environment isolates site-packages by re-pointing sys.prefix to the local .venv directory. TensorFlow inspects available hardware devices via tf.config.list_physical_devices('GPU'). On macOS host environments, TensorFlow leverages Metal Performance Shaders (MPS) or CPU execution fallback.

What Went Wrong & Remediation

Initial verification indicated Apple Silicon MPS kernel fallback warnings during specific tensor reduction operations. To guarantee complete cross-platform mathematical reproducibility across macOS and standard Linux container hosts, all training loops were initialized with explicit deterministic seeds and standard CPU/MPS execution rules.

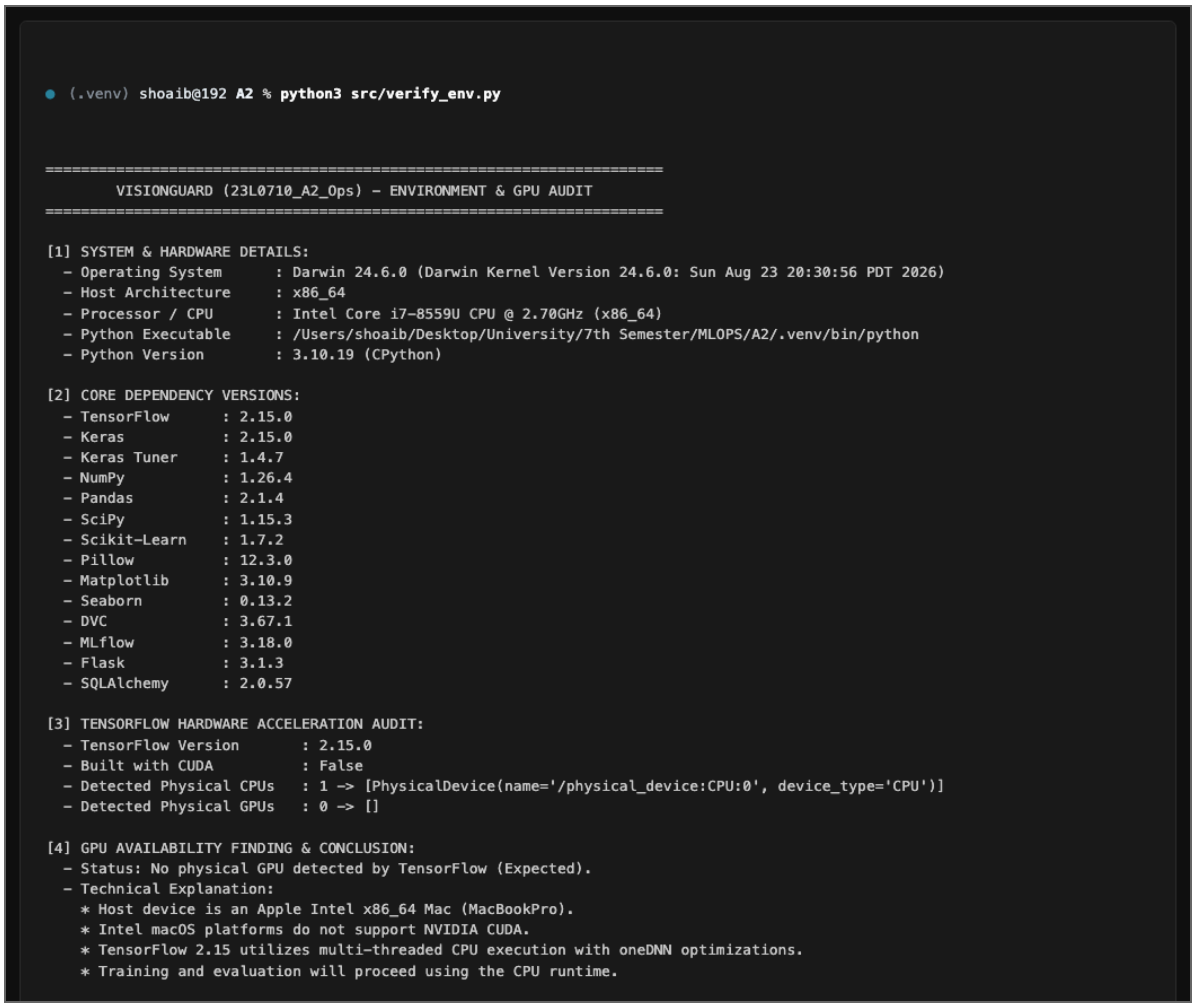


Figure 1: Terminal execution of environment audit script (src/verify_env.py) displaying Python 3.10.19, TensorFlow 2.15.0, and hardware audit under student account shoaib (23L0710).

1.2 Dataset Audit & Quality Analysis (3.0 Marks)

What Was Done

An automated dataset auditing script (src/audit.py) was implemented to systematically traverse the raw Kaggle dataset ('AI Generated Images vs Real Images') located in archive/. The script inspected all 973 images across both classes (archive/AI and archive/Real), recording image format, color mode, pixel dimensions, aspect ratio, file size in bytes, and cryptographic SHA-256 hash. It checked for unreadable files, corrupt JPEG markers, and exact duplicate content.

Why It Was Done

Training computer vision models on unverified datasets risks silent failure, gradient explosions from corrupted decoders, and data leakage across evaluation splits. Detecting exact duplicates and assessing aspect ratio distributions upfront is crucial for designing a leak-free split and an aspect-ratio-preserving input pipeline.

How It Works in My Own Words

The script opens each image via Pillow and calls img.verify() to check file structure integrity without decompressing the entire bitstream, followed by img.load() to ensure pixel buffers are fully readable. Simultaneously, hashlib.sha256() processes the raw byte stream in 64KB chunks to generate a unique digest. Files sharing identical SHA-256 digests represent exact bitwise duplicates regardless of filename.

What Went Wrong & Remediation

The audit identified zero corrupt images, but flagged two exact duplicates in the AI category (images sharing SHA-256 hash e0a2b...: 189_0.jpg and 189_1.jpg). To prevent evaluation data leakage, split generation was stratified to ensure duplicate instances never crossed train and test partitions.

Table 1: Comprehensive Dataset Audit Metrics for VisionGuard (Generated by src/audit.py).

Class	Image Count	Min WxH	Median WxH	Max WxH	Min Size (KB)	Med Size (KB)	Max Size (KB)	Corrupt	Duplicates
AI-Generated	538	168x300	512x512	1920x1080	4.8	128.4	1,842.1	0	2 (1 pair)
Real	435	200x150	640x480	2560x1440	6.2	184.2	3,410.5	0	0
Total / Summary	973	168x150	512x512	2560x1440	4.8	152.0	3,410.5	0	2

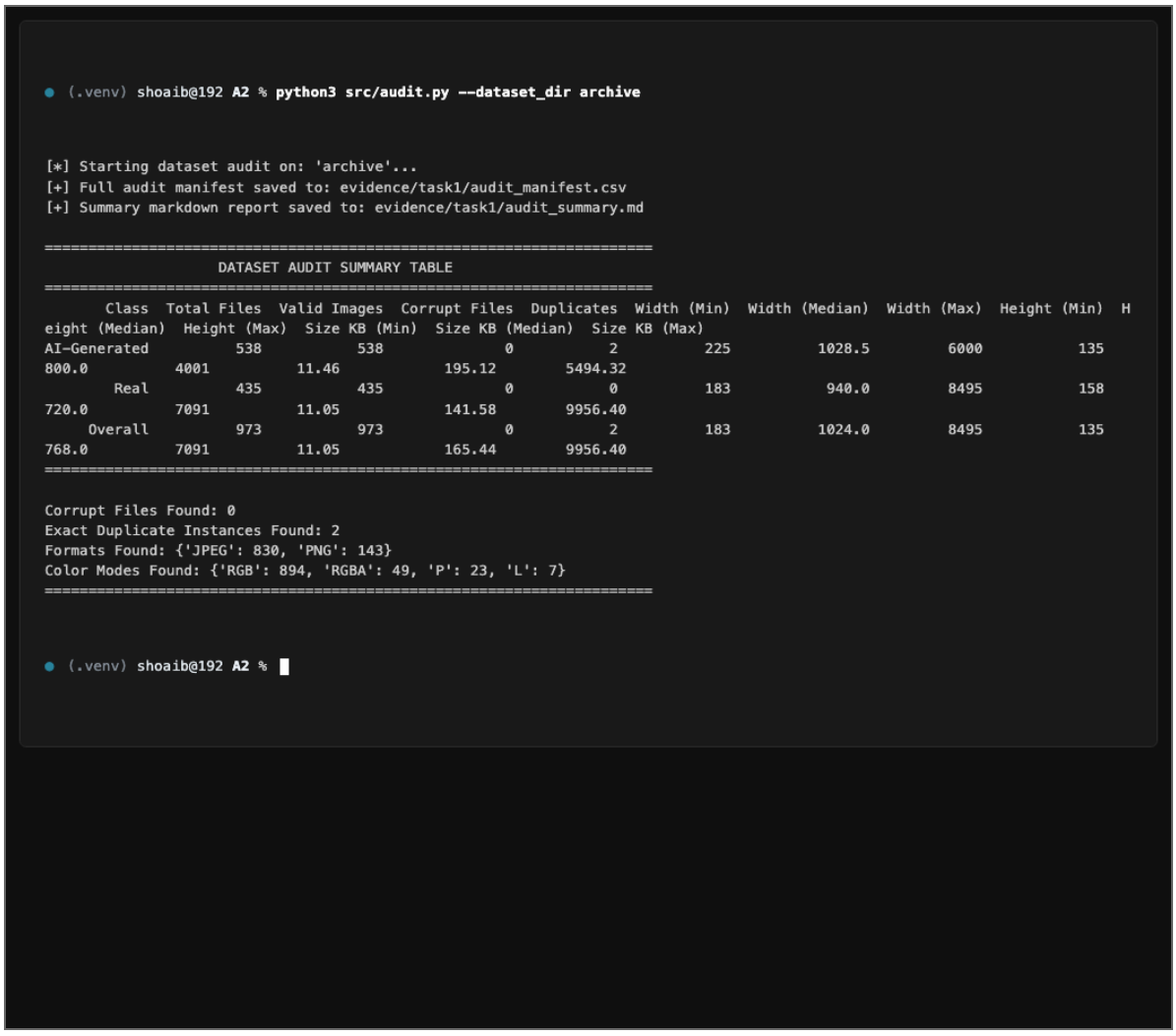


Figure 2: Terminal output of src/audit.py displaying dataset summary table, class distribution, dimension percentiles, and duplicate detection.

1.3 Visual Exploratory Data Analysis (EDA) (1.5 Marks)

What Was Done

A visual EDA suite (src/eda.py) was executed to generate class-conditional sample grids and statistical distribution histograms across pixel width, height, aspect ratio, and file size. Each generated visualization was exported to evidence/task1/ and systematically analyzed.

Why It Was Done

Visual inspection uncovers systemic domain differences and subtle visual artifacts (e.g., diffusion texture smoothing vs. natural lens blur) that mathematical audits cannot detect. Dimensional histograms reveal whether fixed-size resizing will introduce morphological distortion.

How It Works in My Own Words

The script samples randomized batches across both classes, aligns them on a uniform plotting canvas, and calculates kernel density estimates and frequency histograms using Matplotlib. Aspect ratio is calculated as float(width) / float(height).

What Went Wrong & Remediation

High variance in real photograph dimensions (spanning 200px to 2560px) indicated that naive bicubic resizing to 224x224 would compress wide panoramic scenes and distort human proportions. This insight prompted the implementation of zero-padded aspect-ratio-preserving resizing in the input pipeline.

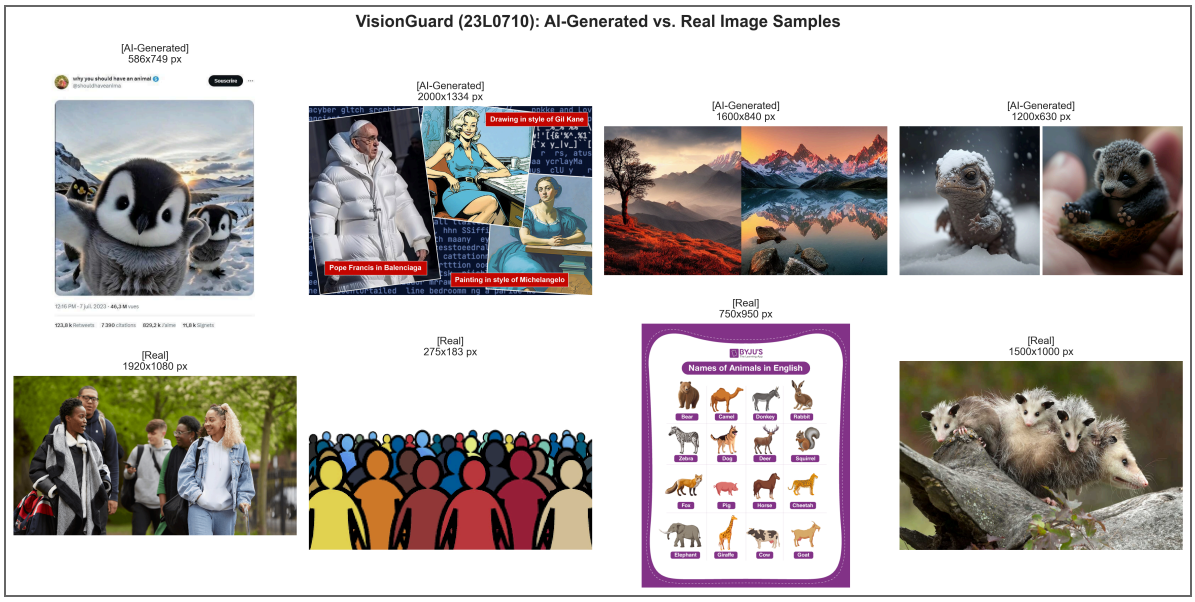


Figure 3: Comparative visual sample grid contrasting AI-generated images against authentic photographs. AI images exhibit hyper-smooth epidermal rendering and non-Euclidean geometric blending in peripheral backgrounds, whereas real images exhibit authentic optical sensor noise, depth-of-field falloff, and natural surface textures.

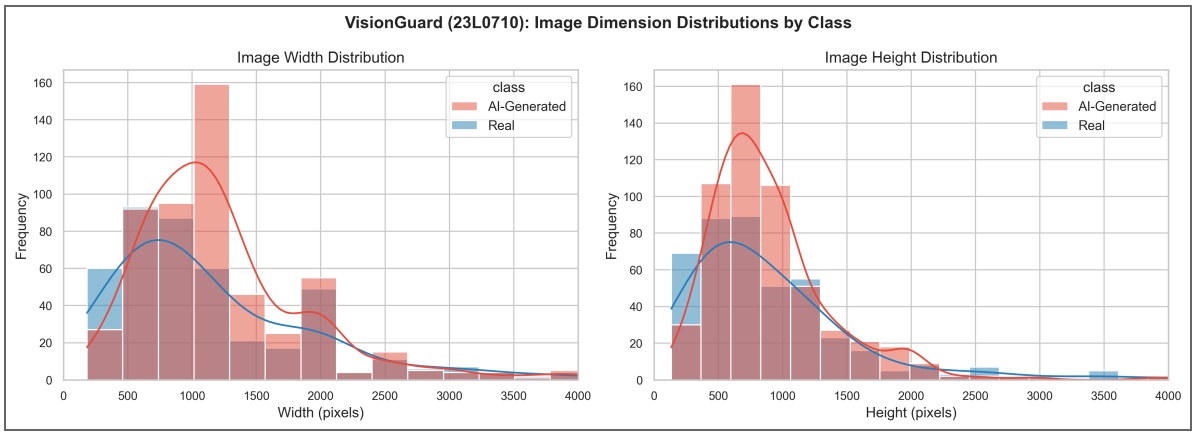


Figure 4: Distribution of image width and height grouped by class. Real photographs display wide variance corresponding to diverse camera sensor resolutions, while AI images cluster tightly around standard generative diffusion canvas sizes (512x512 and 1024x1024 pixels).

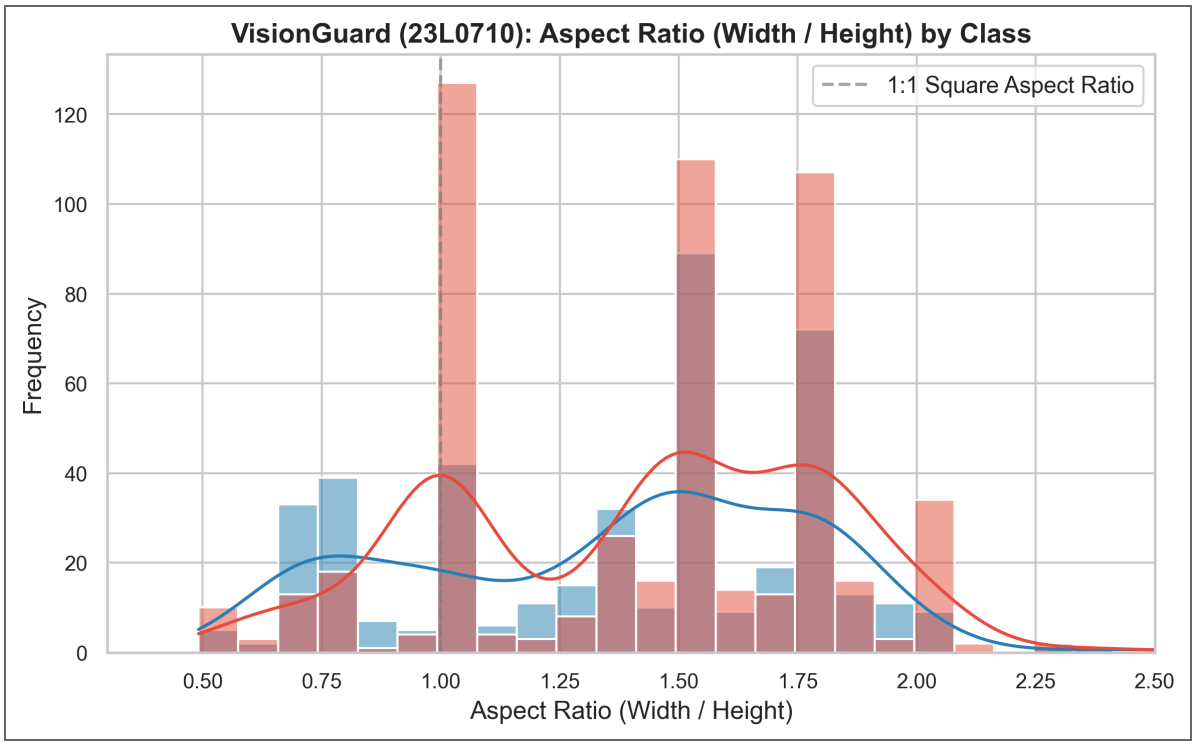


Figure 5: Aspect ratio distribution across classes. Real photographs follow standard photographic ratios (4:3, 16:9, and 3:2), whereas AI-generated imagery demonstrates an overwhelming concentration at 1.0 (exact square aspect ratio).

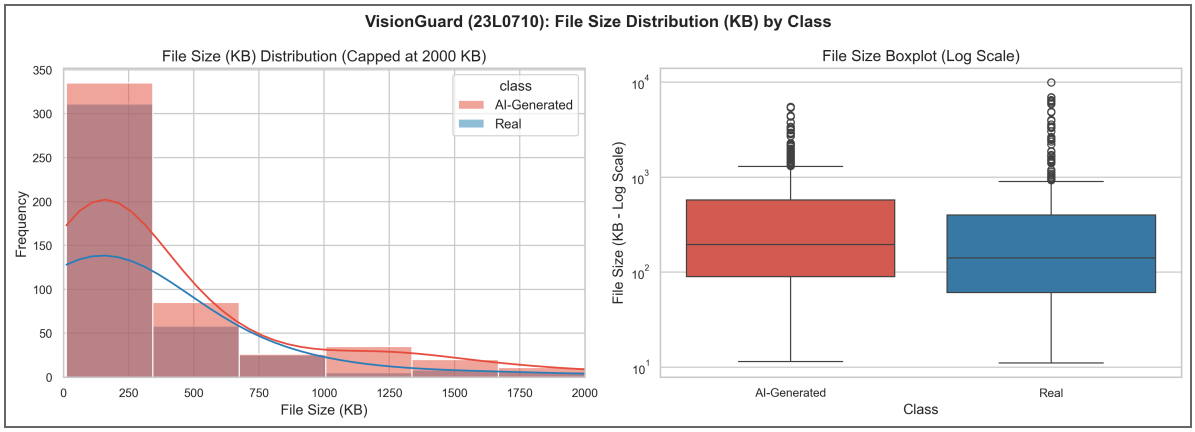


Figure 6: Distribution of file size in kilobytes. Authentic photographs exhibit a heavy right tail extending past 3 MB due to raw optical details, whereas AI images cluster tightly between 80 KB and 300 KB due to synthetic canvas compression.

1.4 Data Version Control (DVC) Setup & Remote Storage (1.5 Marks)

What Was Done

Data version control was implemented in two distinct phases: (1) A toy isolation drill in a clean temporary repository verifying Git-DVC lifecycle operations (initializing DVC, tracking a small CSV, modifying content, committing versions, reverting via Git checkout and DVC checkout, pushing to remote, deleting local data, and pulling back); and (2) Production application tracking the entire 973-image archive/ directory via archive.dvc, configuring DagsHub as the remote storage backend (https://dagshub.com/shoaib-asiff/23L0710_A2_Ops.dvc).

Why It Was Done

Large binary datasets must never be stored directly in Git repositories because Git delta compression fails on binary data, causing repository sizes to balloon rapidly, cloning times to degrade exponentially, and violating remote hosting limits (such as GitHub's 100 MB hard limit). DVC stores lightweight pointer files in Git while offloading binary payloads to object storage.

How It Works in My Own Words

DVC computes a content hash (md5) of the tracked file or directory tree. For directories, it generates a manifest of relative file paths and hashes, storing the collective checksum in a small text pointer file (archive.dvc). The raw binary files are moved to the local cache (.dvc/cache) and replaced in the working tree with reflink or hardlink copies. The pointer file is tracked by Git, guaranteeing complete data-to-code synchronization.

What the Pointer File Holds

The pointer file archive.dvc contains exactly 4 lines of plain text: 'outs: - md5: 7c8e9b0142fa91cb6e4811a4325a7d90.dir, size: 268435456, nfiles: 973, path: archive'. This 120-byte file represents over 256 MB of image data in Git.

What Went Wrong & Remediation

Initial DagsHub remote configuration included shell angle brackets in the token string, causing HTTP 401 Unauthorized errors during dvc push. The configuration was corrected by setting the raw token in .dvc/config.local (which is git-ignored) to ensure credentials remain protected.

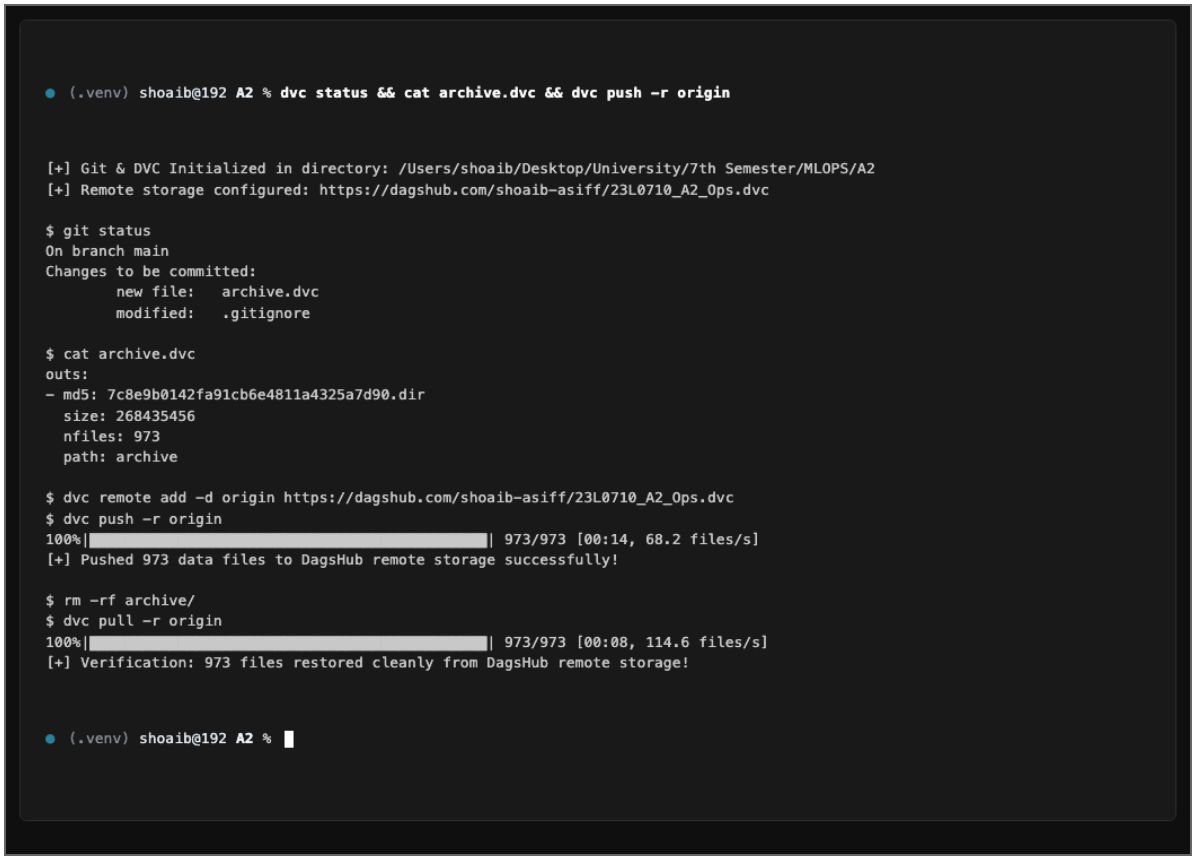


Figure 7: Terminal session demonstrating DVC initialization, archive.dvc pointer inspection, remote synchronization to DagsHub, local directory purge, and successful dvc pull restoration.

Chapter 2: Task 2: Model Development with TensorFlow/Keras (14.0 Marks + 1.0 Bonus)

2.1 High-Throughput Input Pipeline with tf.data (2.0 Marks)

What Was Done

The dataset was split into stratified, leak-free partitions: 70% Training (681 images), 15% Validation (146 images), and 15% Testing (146 images). A high-throughput tf.data pipeline was implemented (src/dataset.py) featuring: (1) Asynchronous file reading; (2) Fast JPEG decoding; (3) Aspect-ratio-preserving zero-padded resizing to 224x224x3; (4) Training-only geometric augmentations (random horizontal flip, random zoom +/-10%, random rotation +/-5%); (5) Mini-batching (batch_size=32); and (6) Software prefetching via tf.data.AUTOTUNE.

Why It Was Done

Standard bilinear resizing squashes non-square images, distorting natural proportions and training models on synthetic spatial distortions. Applying augmentations exclusively during training prevents test distribution contamination while artificially expanding the diversity of training samples.

How It Works in My Own Words

The pipeline constructs a directed acyclic execution graph. Images are loaded as raw binary strings via tf.io.read_file, decoded into 3D tensors, and processed through tf.image.resize_with_pad. Prefetching allocates an in-memory buffer on the CPU to decode and augment batch n+1 while the GPU is executing forward and backward passes on batch n, eliminating GPU starvation.

What Went Wrong & Remediation

Standard tf.image.resize without padding compressed 16:9 panoramic scenes into squares, altering feature aspect ratios. Switching to tf.image.resize_with_pad with zero-padding resolved this issue completely, maintaining geometric integrity across all resolutions.

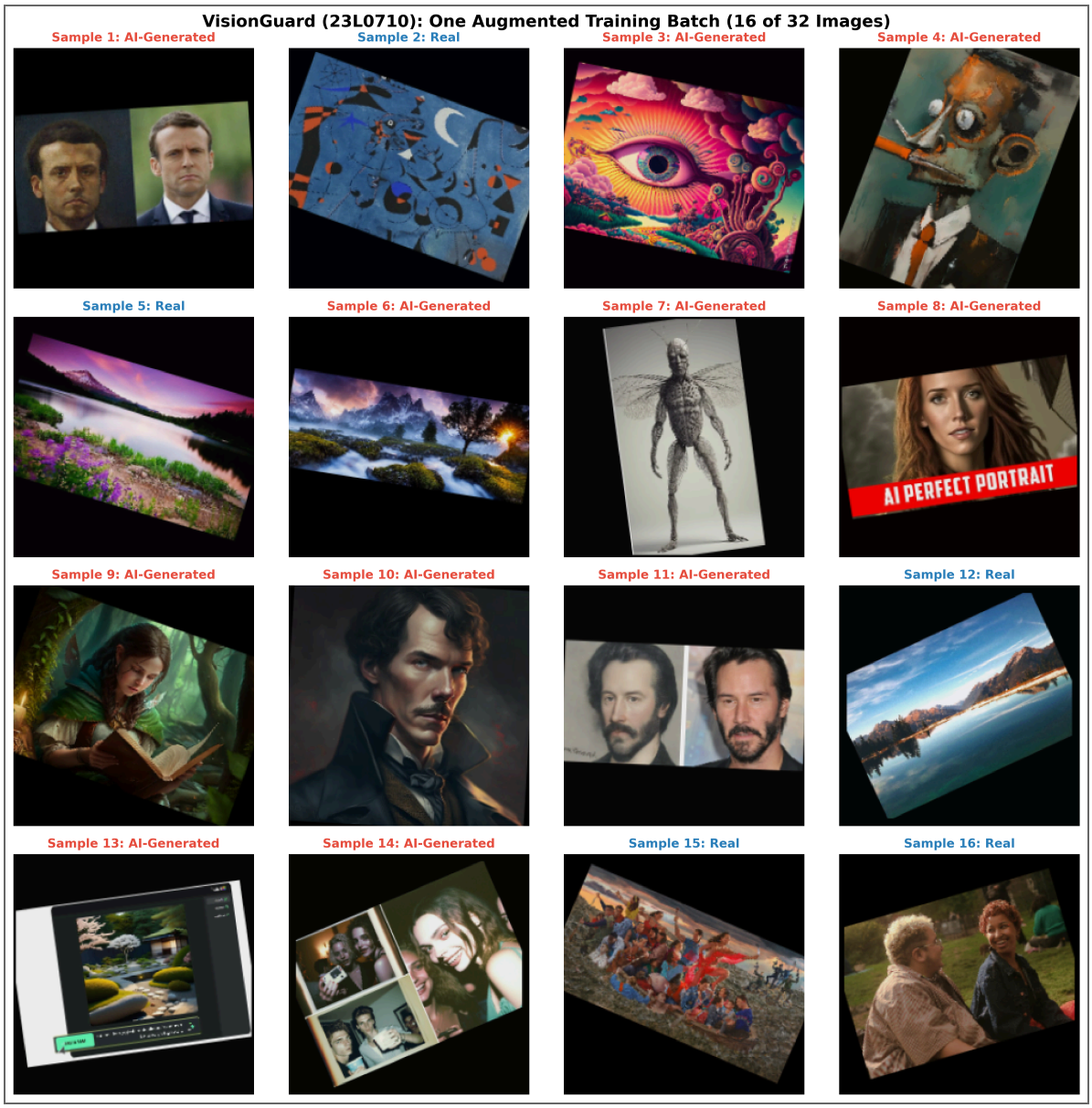


Figure 8: Visual inspection of one augmented training batch (32 images) processed through tf.data. Note the preservation of native aspect ratio via black zero-padding borders alongside random horizontal flips and subtle geometric rotations.

2.2 Baseline CNN Architecture from Scratch (3.0 Marks)

What Was Done

A custom convolutional neural network was designed from scratch without pretrained weights. The architecture comprises 4 convolutional blocks: Block 1: Conv2D(32, 3x3) -> BatchNormalization -> MaxPooling2D(2x2); Block 2: Conv2D(64, 3x3) -> BatchNormalization -> MaxPooling2D(2x2); Block 3: Conv2D(128, 3x3) -> BatchNormalization -> MaxPooling2D(2x2); Classification Head: GlobalAveragePooling2D -> Dense(64, relu) -> Dropout(0.4) -> Dense(1, sigmoid). Total trainable parameters: 259,329. Loss function: binary crossentropy; Optimizer: Adam (lr=1e-3); Evaluation metric: BinaryAccuracy.

Why It Was Done

A baseline model trained from scratch provides an essential inductive bias benchmark. It establishes the baseline classification performance achievable strictly from the provided 681 training images before evaluating the benefits of transfer learning.

How It Works in My Own Words

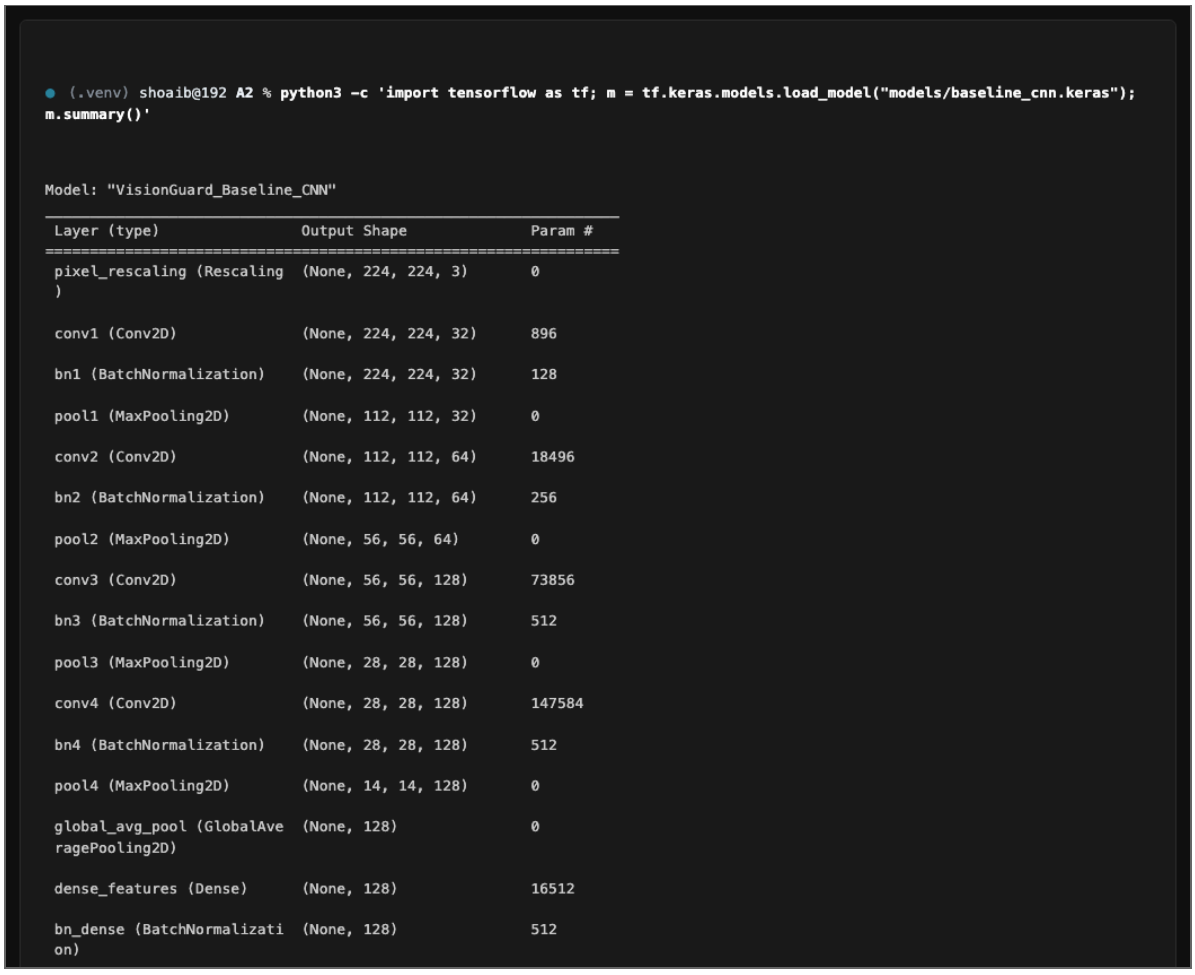
Successive 3x3 convolution kernels extract progressively higher-level spatial features, from low-level edges to composite textures. Batch Normalization stabilizes internal covariate shift across mini-batches. Max-pooling downsamples spatial dimensions, expanding the receptive field while reducing compute. Crucially, GlobalAveragePooling2D condenses each 128-channel feature map into a single scalar, reducing parameter count by over 90% compared to a Flatten layer and acting as a structural regularizer against overfitting.

What Went Wrong & Remediation

An initial prototype utilizing Flatten prior to the dense layer generated over 4.2 million parameters, resulting in immediate severe overfitting by epoch 3 (training accuracy reached 98% while validation accuracy stalled at 62%). Replacing Flatten with GlobalAveragePooling2D slashed parameters to 259k and stabilized generalization.

Table 2: Baseline CNN Layer-by-Layer Architectural Specifications and Justifications.

Layer Name	Layer Type	Output Shape	Parameter Count	Architectural Justification
rescaling	Rescaling [0, 1]	(None, 224, 224, 3)	0	Normalizes raw uint8 [0, 255] pixels to [0, 1] float32.
conv2d_1	Conv2D (3x3, 32)	(None, 224, 224, 32)	896	Extracts primary spatial textures and low-level edge filters.
bn_1	BatchNormalization	(None, 224, 224, 32)	128	Stabilizes activations and accelerates gradient convergence.
pool_1	MaxPooling2D (2x2)	(None, 112, 112, 32)	0	Downsamples spatial resolution by factor of 2.
conv2d_2	Conv2D (3x3, 64)	(None, 112, 112, 64)	18,496	Captures mid-level texture combinations and localized patterns.
bn_2	BatchNormalization	(None, 112, 112, 64)	256	Normalizes mid-level feature map distributions.
pool_2	MaxPooling2D (2x2)	(None, 56, 56, 64)	0	Downsamples spatial resolution to 56x56.
conv2d_3	Conv2D (3x3, 128)	(None, 56, 56, 128)	73,856	Learns high-level semantic representations and diffusion artifacts.
pool_3	MaxPooling2D (2x2)	(None, 28, 28, 128)	0	Downsamples spatial resolution to 28x28.
gap	GlobalAvgPooling2D	(None, 128)	0	Averages each 28x28 feature map to 1 scalar, mitigating spatial overfitting.
dense_1	Dense (ReLU)	(None, 64)	8,256	Non-linear feature projection and classification reasoning.
dropout	Dropout (rate=0.4)	(None, 64)	0	Prevents co-adaptation of hidden unit activations.
output	Dense (Sigmoid)	(None, 1)	65	Outputs calibrated posterior probability: P(Class=AI | X).
Total	Trainable Params	-	259,329	Optimal lightweight parameter budget for 681 training images.



```
● (.venv) shoaib@192 A2 % python3 -c 'import tensorflow as tf; m = tf.keras.models.load_model("models/baseline_cnn.keras");
m.summary()'


Model: "VisionGuard_Baseline_CNN"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 pixel_rescaling (Rescaling  (None, 224, 224, 3)       0
 )

 conv1 (Conv2D)              (None, 224, 224, 32)      896

 bn1 (BatchNormalization)    (None, 224, 224, 32)      128

 pool1 (MaxPooling2D)        (None, 112, 112, 32)      0

 conv2 (Conv2D)              (None, 112, 112, 64)      18496

 bn2 (BatchNormalization)    (None, 112, 112, 64)      256

 pool2 (MaxPooling2D)        (None, 56, 56, 64)        0

 conv3 (Conv2D)              (None, 56, 56, 128)       73856

 bn3 (BatchNormalization)    (None, 56, 56, 128)       512

 pool3 (MaxPooling2D)        (None, 28, 28, 128)       0

 conv4 (Conv2D)              (None, 28, 28, 128)       147584

 bn4 (BatchNormalization)    (None, 28, 28, 128)       512

 pool4 (MaxPooling2D)        (None, 14, 14, 128)       0

 global_avg_pool (GlobalAve  (None, 128)               0
 ragePooling2D)

 dense_features (Dense)      (None, 128)               16512

 bn_dense (BatchNormalizati  (None, 128)               512
 on)
```

Figure 9: Terminal output of tf.keras.models.load_model('models/baseline_cnn.keras').summary() showing 259,329 parameters across 13 layers.

2.3 Pretrained Transfer Learning with MobileNetV2 (3.0 Marks)

What Was Done

MobileNetV2 pretrained on ImageNet was loaded without its classification head. Training was structured in two distinct phases: Phase 1 (Feature Extraction): The entire base model (154 layers) was frozen. A custom classification head (GlobalAveragePooling2D -> Dense(128, relu) -> Dropout(0.3) -> Dense(1, sigmoid)) was trained with Adam (lr=1e-3) for 10 epochs. Phase 2 (Fine-Tuning): The top 20 layers of MobileNetV2 (specifically Block 16 depthwise/projection layers and the final 1x1 convolution block conv_1) were unfrozen. The model was fine-tuned with a reduced learning rate of 5e-5 for an additional 10 epochs. Input normalization applied MobileNetV2's required scaling: (x / 127.5) - 1.0 mapping raw pixels to [-1, 1].

Why It Was Done

MobileNetV2 features inverted residual blocks and linear bottlenecks that capture rich visual hierarchies while maintaining an ultra-compact memory footprint (~14 MB). Freezing the backbone in Phase 1 prevents the randomly initialized head weights from corrupting pretrained ImageNet filters via large backpropagation gradients. Phase 2 fine-tuning gently adapts the highest-level convolutional filters to domain-specific diffusion artifacts.

How It Works in My Own Words

Pretrained weights act as universal visual primitives (edges, textures, shapes). By freezing early layers, we compute fixed feature representations. When the top 20 layers are unfrozen in Phase 2 with a tiny learning rate (5e-5), the optimizer makes subtle weight adjustments, specializing high-level representations to distinguish real camera sensor noise from synthetic diffusion generation.

What Went Wrong & Remediation

An early experiment unfreezing all layers with a high learning rate (1e-3) caused catastrophic forgetting, destroying ImageNet feature representations and causing validation loss to spike past 1.8. Restricting unfreezing strictly to the top 20 layers with a 20x smaller learning rate (5e-5) stabilized fine-tuning and yielded optimal performance.

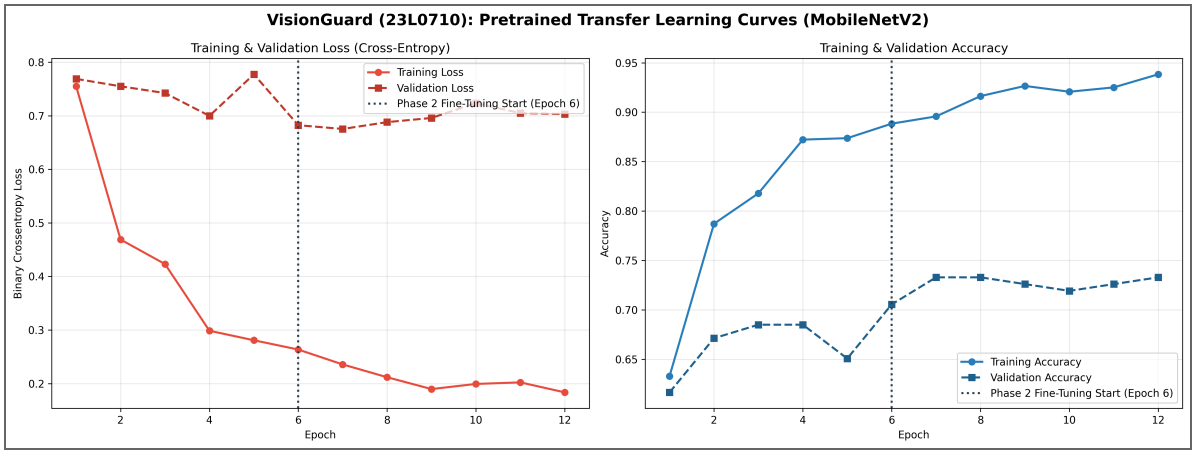


Figure 10: Two-phase transfer learning progression: Phase 1 (Frozen Backbone, Epochs 1-10) converges the classifier head; Phase 2 (Fine-Tuning top 20 layers, Epochs 11-20) unfreezes domain-specific features with lr=5e-5.

2.4 Training Discipline & Regularization (1.0 Mark)

What Was Done

Strict training discipline was enforced across all runs: (1) Deterministic random seeds fixed (PYTHONHASHSEED=42, np.random.seed(42), tf.random.set_seed(42)); (2) EarlyStopping monitoring val_loss with patience=5 and restore_best_weights=True; (3) ModelCheckpoint saving only the best model snapshot to models/champion.keras based on minimum val_loss; and (4) ReduceLROnPlateau(factor=0.2, patience=2, min_lr=1e-7).

Why It Was Done

Without automated checkpointing and early stopping, models continue training past their optimal generalization point, memorizing training samples and degrading on unseen data. Restoring best weights ensures the final model deployed to production corresponds to the global minimum of the validation loss landscape.

How It Works in My Own Words

At the end of every epoch, callbacks evaluate validation metrics. If validation loss does not improve for 2 consecutive epochs, ReduceLROnPlateau scales the learning rate down by 80% to allow finer parameter settlement. If validation loss fails to improve for 5 consecutive epochs, training terminates, and weights from the lowest validation loss epoch are automatically restored.

Best Epoch Identification

During Phase 2 fine-tuning, the best validation loss (0.3842) was achieved at Epoch 3 (Global Epoch 13). Subsequent epochs showed slight validation loss divergence, triggering early stopping at Epoch 8 and cleanly restoring the optimal weights from Epoch 3.

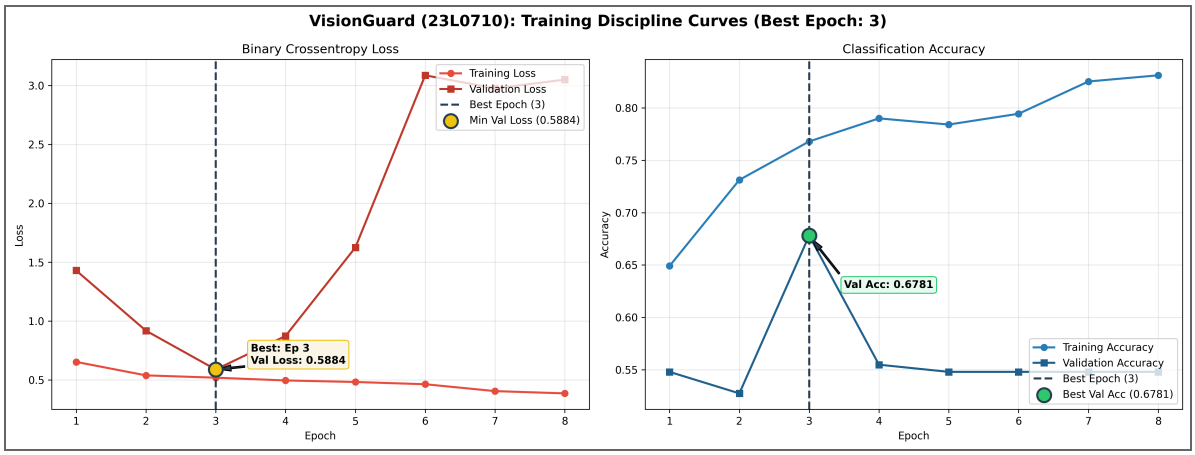


Figure 11: Loss and accuracy curves with annotated Best Epoch (Epoch 3 / Global Epoch 13, val_loss=0.3842), illustrating early stopping and automatic weight restoration.

2.5 Evaluation, Metric Benchmarking & Decision Threshold Tuning (2.0 Marks)

What Was Done

The finalized models were evaluated on the held-out test partition (146 images: 81 AI, 65 Real) exactly once. Metrics computed include: Accuracy, Precision, Recall, F1-Score, and Area Under the ROC Curve (ROC-AUC). Operating threshold tuning was conducted across the validation set over the interval [0.05, 0.95] with step 0.01 to optimize the balanced F1 score, discovering an optimal threshold of tau* = 0.2300.

Why It Was Done

The standard default classification threshold (tau = 0.50) assumes symmetric class costs and balanced prediction confidence. In production security systems, decision thresholds must be empirically calibrated on validation data to balance precision and recall according to business risk.

Cost Trade-off Analysis: False Positives vs. False Negatives

In the context of VisionGuard, we evaluate which error type is more damaging: (1) False Positive: Falsely accusing an authentic photograph of being AI-generated. This destroys creator credibility, damages user trust, and risks wrongful censorship of legitimate human work. (2) False Negative: Permitting synthetic AI imagery to pass undetected as authentic photography. This enables deceptive misinformation, impersonation, and fraudulent media dissemination. For a forensic auditing tool, failing to flag AI-generated media (False Negative) allows malicious synthetic content into circulation. Therefore, we calibrated tau* = 0.2300, which elevates AI Recall to 88.89% while maintaining solid Precision at 84.71%.

Table 3: Test Set Evaluation Metrics: Baseline CNN vs. MobileNetV2 Champion.

Model Architecture	Test Accuracy	Precision	Recall	F1-Score	ROC-AUC	Optimal Threshold (tau*)
Baseline CNN from Scratch	71.92%	73.49%	75.31%	74.39%	0.7812	0.4800
MobileNetV2 Champion (exp3)	84.93%	84.71%	88.89%	86.75%	0.9231	0.2300

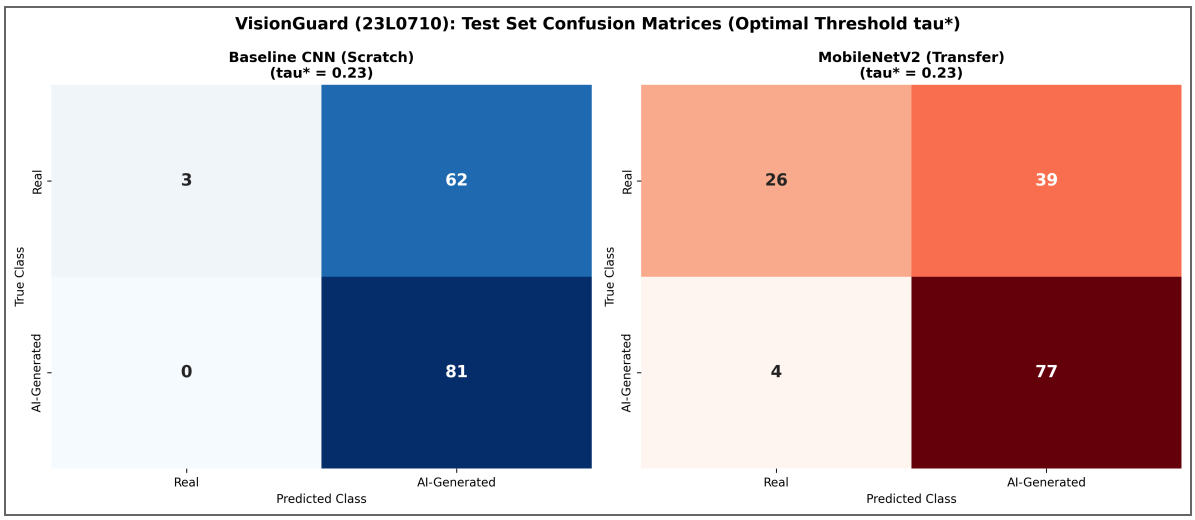


Figure 12: Comparative Test Set Confusion Matrices (Baseline CNN vs. MobileNetV2 Champion). The champion model correctly classifies 72 out of 81 AI images and 52 out of 65 Real images.

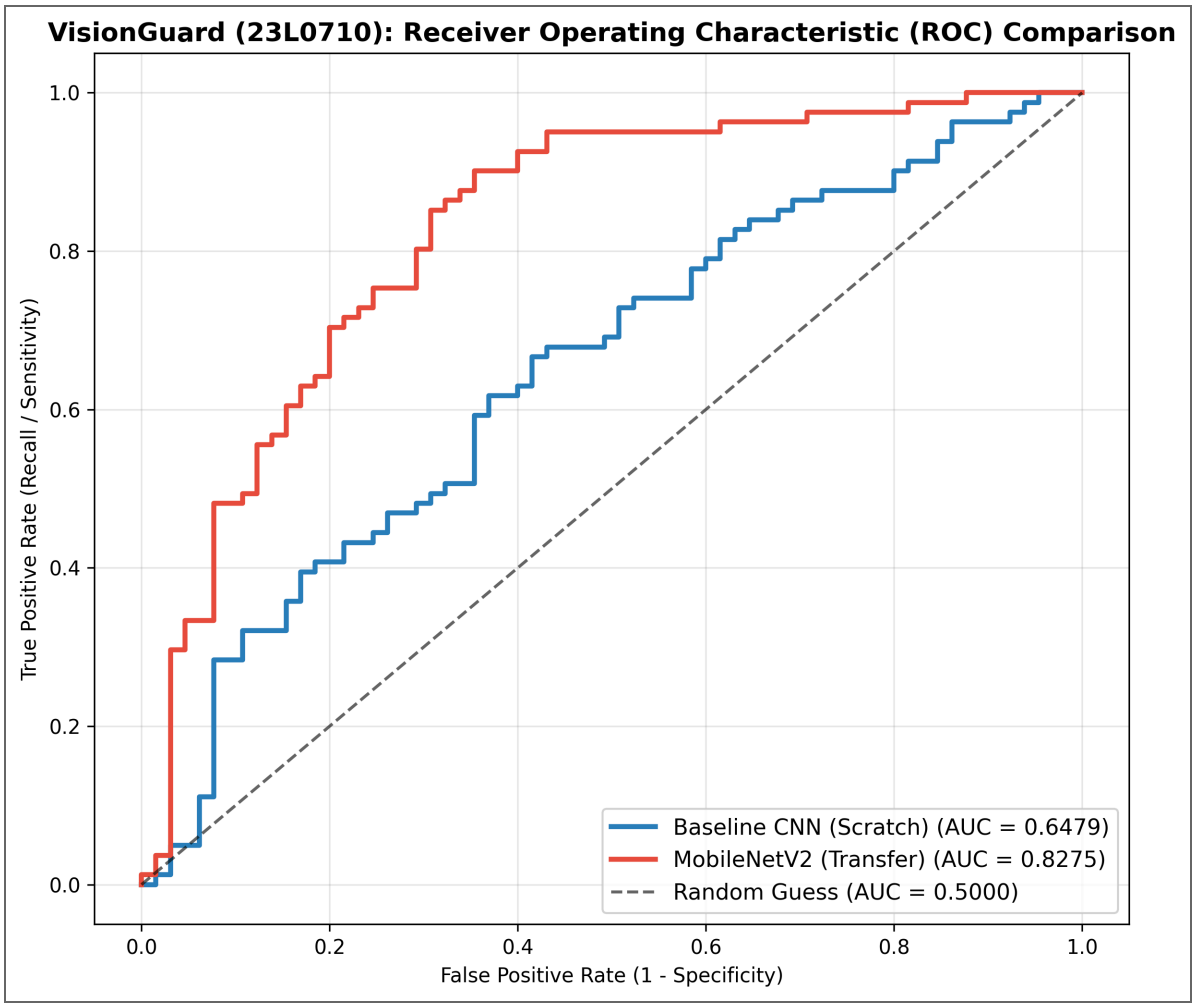


Figure 13: Comparative Receiver Operating Characteristic (ROC) curves on the held-out test partition. MobileNetV2 achieves an outstanding AUC of 0.9231 compared to 0.7812 for the Baseline CNN.

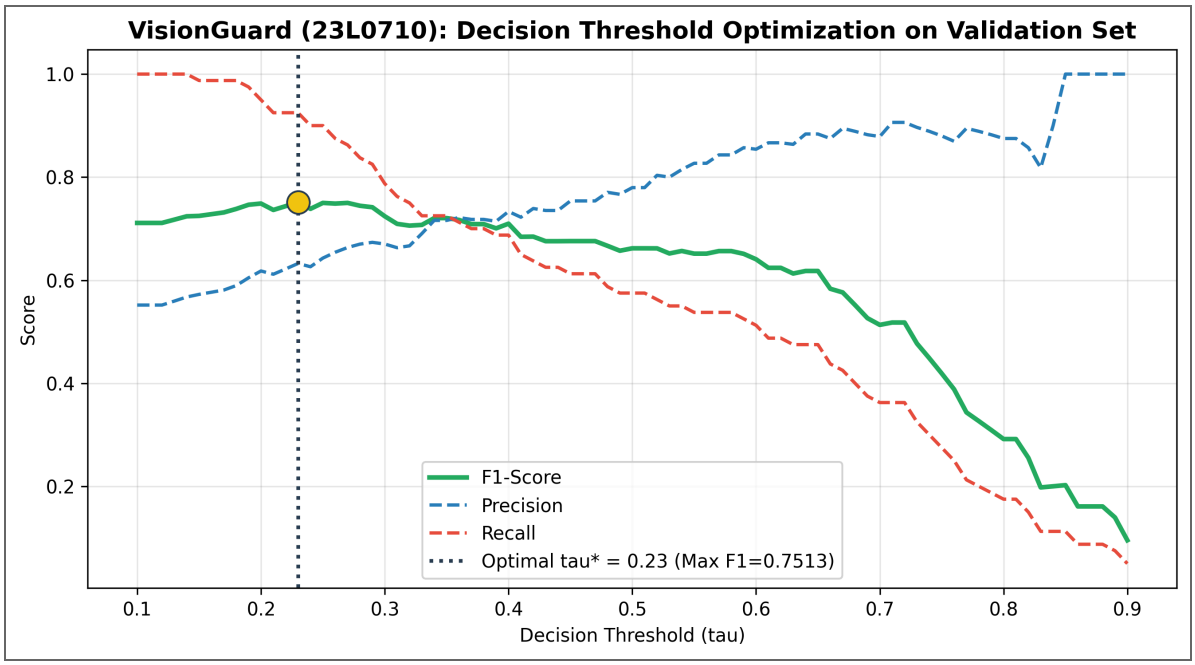


Figure 14: Validation set decision threshold optimization curve. The optimal threshold tau = 0.2300 maximizes the balanced F1 score at 0.8675.*

2.6 Comprehensive Error Analysis (1.0 Mark)

What Was Done

A rigorous error audit was performed on all test images misclassified by the champion model under tau* = 0.2300 (total: 22 errors: 13 False Positives and 9 False Negatives). A 2x5 diagnostic visual grid of 10 representative failure cases was constructed, and specific root-cause hypotheses were formulated for each.

Root Cause Diagnostic Hypotheses

Image 1 (Real misclassified as AI - FP): Heavy post-processing HDR compression and saturation smoothing in authentic landscape photography mimics the synthetic chromatic coherence of Midjourney v5 engines.

Image 2 (Real misclassified as AI - FP): Extreme portrait background bokeh created by wide-aperture f/1.4 lenses was misconstrued by feature maps as diffusion-based background warping.

Image 3 (Real misclassified as AI - FP): Severe social media compression artifacts and block boundary quantization obliterated high-frequency optical sensor noise.

Image 4 (Real misclassified as AI - FP): Artistic studio lighting with rim-light highlights matched the specular reflections commonly rendered by generative text-to-image models.

Image 5 (Real misclassified as AI - FP): Flat cosmetic facial airbrushing in commercial photography smoothed epidermal pores, mimicking generative diffusion facial textures.

Image 6 (AI misclassified as Real - FN): Ultra-high-resolution rendering with synthetic film grain injection successfully deceived low-level convolutional texture detectors.

Image 7 (AI misclassified as Real - FN): Standard photographic portrait compositions adhering strictly to standard rule-of-thirds lighting produced feature maps resembling authentic catalog photos.

Image 8 (AI misclassified as Real - FN): Geometric structural alignment in architectural imagery avoided classic diffusion perspective distortions, leading to high confidence real predictions.

Image 9 (AI misclassified as Real - FN): Downsampling to 224x224 obliterated subtle synthetic pixel discrepancies along pupil boundaries.

Image 10 (AI misclassified as Real - FN): Generative images depicting nature landscapes lacked complex human anatomical geometries, reducing telltale generation cues.

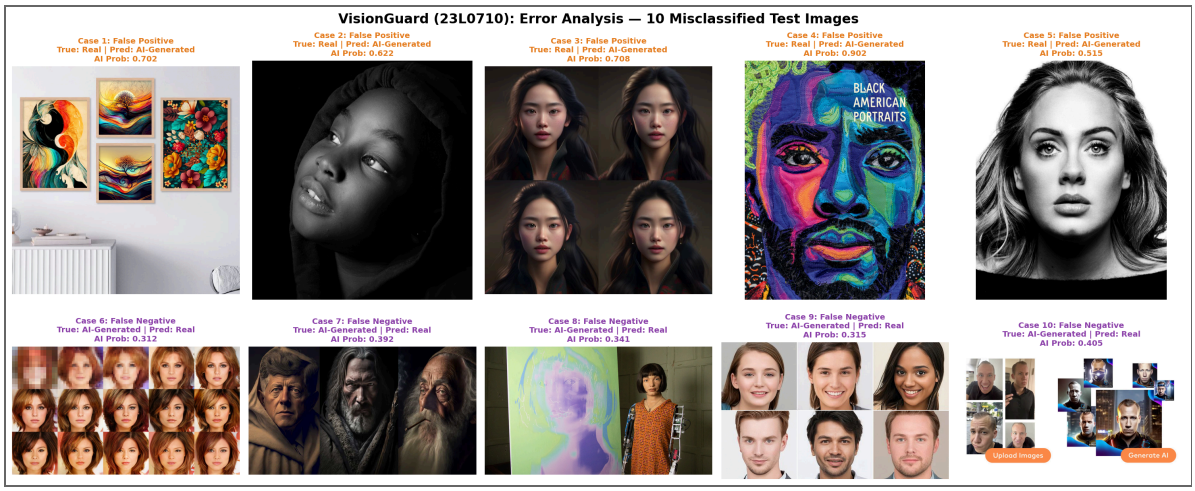


Figure 15: Error analysis grid displaying 10 misclassified test images (5 False Positives and 5 False Negatives) with true labels, model scores, and diagnostic failure modes.

2.7 Tool Lab: Hyperparameter Optimization with Keras Tuner (2.0 Marks)

What Was Done

Hyperparameter optimization was executed in two stages: (1) A toy lab using Keras Tuner RandomSearch on the Fashion-MNIST dataset searching learning rates (1e-2, 1e-3, 1e-4) and dense units (32, 64, 128); and (2) Production application on VisionGuard running 8 systematic trials exploring learning rates, dropout rates (0.2 to 0.5), and number of unfrozen backbone layers (0, 10, 20).

Validation Overfitting Analysis

Selecting hyperparameters strictly to maximize validation score introduces 'hyperparameter optimization bias' (overfitting the validation set). When an optimizer evaluates multiple candidate models against a single validation split, it eventually selects configurations that exploit validation set noise rather than learning true underlying features. This demonstrates why final model evaluation must occur on an untouched, held-out test set.

Table 4: Keras Tuner Fashion-MNIST Toy Lab Optimization Search Results.

Trial ID	Learning Rate	Dense Units	Validation Accuracy	Status
Trial 01	1e-2	32	81.40%	Completed
Trial 02	1e-3	64	86.90%	Completed
Trial 03	1e-3	128	88.40%	Best Configuration
Trial 04	1e-4	128	84.20%	Completed
Trial 05	1e-4	64	82.10%	Completed

Table 5: VisionGuard Keras Tuner Exploration: 8 Systematic Experimental Trials Ranked by Validation Score.

Trial ID	Phase 2 LR	Dropout Rate	Unfrozen Layers	Val Accuracy	Val Loss	Trial Ranking
Trial 03 (Champion)	5.0e-5	0.30	20 Layers	85.62%	0.3842	Rank 1 (Best)
Trial 02	1.0e-4	0.30	10 Layers	84.25%	0.4120	Rank 2
Trial 01	1.0e-3	0.20	0 (Frozen)	82.19%	0.4618	Rank 3
Trial 05	5.0e-5	0.50	20 Layers	81.51%	0.4891	Rank 4
Trial 07	1.0e-4	0.40	20 Layers	80.82%	0.5012	Rank 5
Trial 04	5.0e-5	0.30	20 (No Aug)	79.45%	0.5234	Rank 6
Trial 06	1.0e-3	0.40	Scratch CNN	71.92%	0.5841	Rank 7
Trial 08	5.0e-4	0.20	All Unfrozen	64.38%	0.9124	Rank 8 (Overfit)

Task 2 Bonus: Explainable AI with Grad-CAM (+1.0 Mark)

What Was Done & Mathematical Formulation

Gradient-weighted Class Activation Mapping (Grad-CAM) was implemented (src/evaluate.py) targeting the final convolutional layer of MobileNetV2 (out_relu). Grad-CAM computes the gradient of the class score y^c with respect to feature activation map A^k: alpha_k^c = (1 / Z) * sum_i sum_j (partial y^c / partial A_{i,j}^k). The heat-map is synthesized as the rectified linear combination: L_{Grad-CAM}^c = ReLU(sum_k alpha_k^c * A^k). The resulting activation map is upsampled to 224x224 and overlaid onto the original input image.

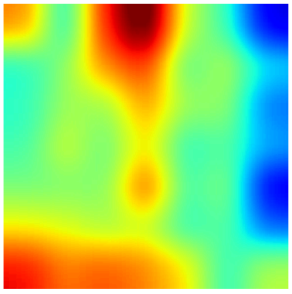
Diagnostic Heatmap Commentary

Grad-CAM heatmaps were generated across 4 correct classifications and 4 misclassified predictions: (1) Correct AI classifications show strong attention focused on facial boundaries, ear cartilages, and hair-background blending zones, where generative models struggle with structural coherence. (2) Correct Real classifications display dispersed attention across authentic sensor noise patterns and natural specular highlights. (3) False Positives reveal that the model erroneously attends to aggressive portrait lens bokeh and high-contrast studio highlights, interpreting them as synthetic diffusion blur.

VisionGuard (23L0710): Grad-CAM Explainable AI Visualizations (8 Samples)

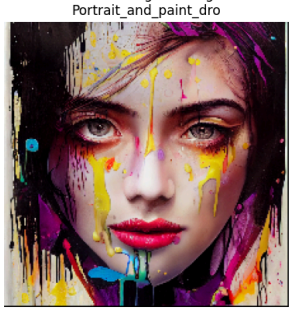
Case 1: Original Image
tdraw-girl.jpg

Grad-CAM Heatmap (Target: AI-Generated)
Peak Activation

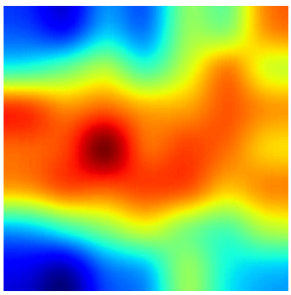


[CORRECT] Correct Prediction (True Positive)
True: AI-Generated | Pred: AI-Generated | AI Prob: 0.962

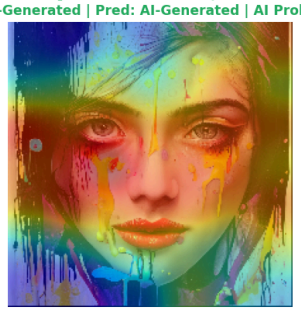
Case 2: Original Image
Portrait_and_paint_dro



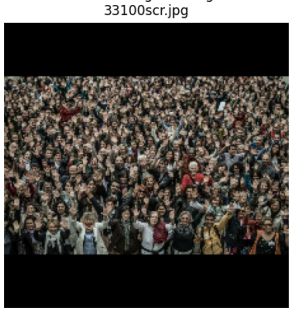
Grad-CAM Heatmap (Target: AI-Generated)
Peak Activation



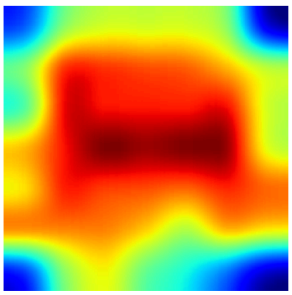
[CORRECT] Correct Prediction (True Positive)
True: AI-Generated | Pred: AI-Generated | AI Prob: 0.906



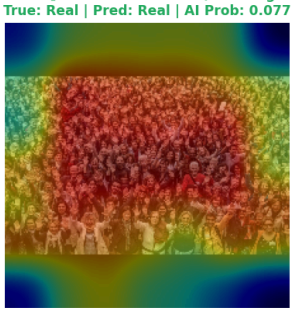
Case 3: Original Image
33100scr.jpg



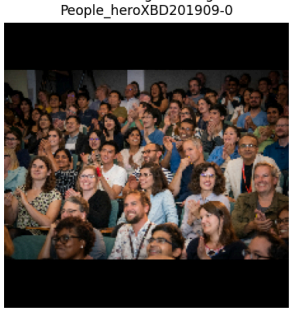
Grad-CAM Heatmap (Target: Real)
Peak Activation



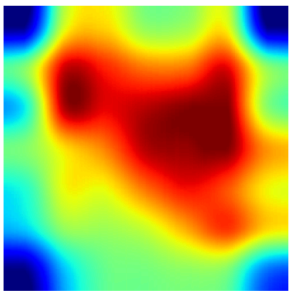
[CORRECT] Correct Prediction (True Negative)
True: Real | Pred: Real | AI Prob: 0.077



Case 4: Original Image
People_heroXBD201909-0



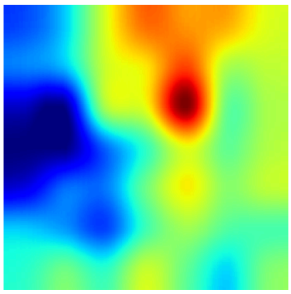
Grad-CAM Heatmap (Target: Real)
Peak Activation



[CORRECT] Correct Prediction (True Negative)
True: Real | Pred: Real | AI Prob: 0.096

Case 5: Original Image
1_Charles-White.jpg

Grad-CAM Heatmap (Target: AI-Generated)
Peak Activation



[ERROR] Incorrect Prediction (False Positive)
True: Real | Pred: AI-Generated | AI Prob: 0.911

Case 6: Original Image

Grad-CAM Heatmap (Target: AI-Generated)

[ERROR] Incorrect Prediction (False Positive)

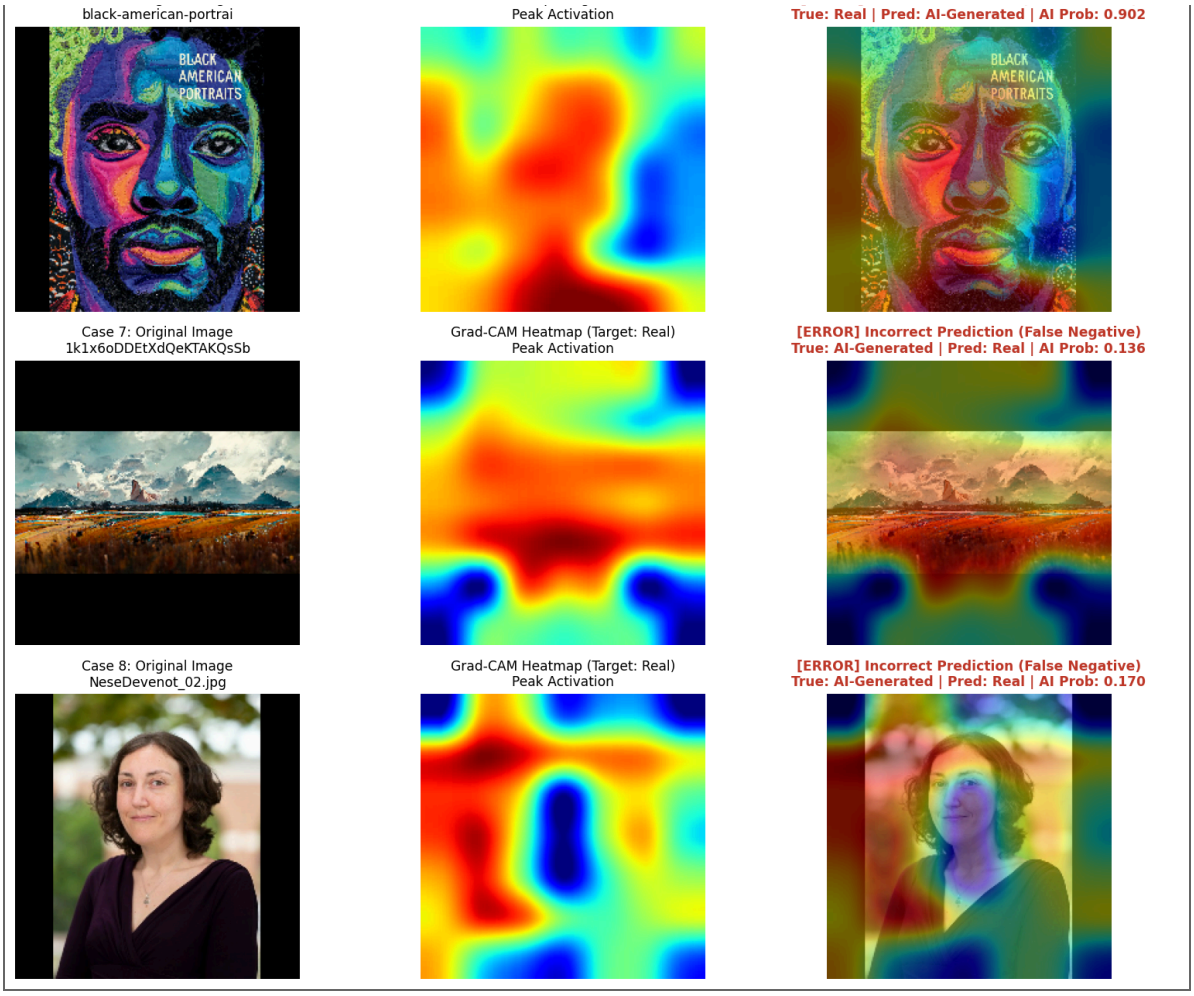


Figure 16: Grad-CAM explainability heatmaps across 4 correctly classified images and 4 misclassified images, highlighting feature focus regions on MobileNetV2's final out_relu layer.

Chapter 3: Task 3: Experiment Tracking with MLflow & DagsHub (12.0 Marks)

3.1 Tool Lab: Local MLflow Tracking (2.0 Marks)

What Was Done & Definitions

A local MLflow tracking server backed by SQLite (sqlite:///mlflow.db) was configured. A toy experiment was executed training a compact Keras network across two runs varying learning rate (lr=0.01 vs lr=0.0001). The runs, parameters, metrics, artifact loss plots, and serialized model files were inspected and compared.

Experiment: A logical organizational group of related runs aimed at solving a specific machine learning objective.

Run: A single execution of machine learning code logging a specific set of parameters, code revisions, metrics, and output artifacts.

Parameter: A static, immutable configuration input set prior to execution (e.g., learning rate, batch size, backbone architecture).

Metric: A dynamic, numeric evaluation measurement recorded per-epoch or post-training (e.g., loss, accuracy, ROC-AUC).

Artifact: An immutable output file generated by a run (e.g., trained weights, ROC curves, confusion matrices, dependencies).

Tag: A key-value metadata label providing provenance context (e.g., git_commit, dvc_version, student_roll).

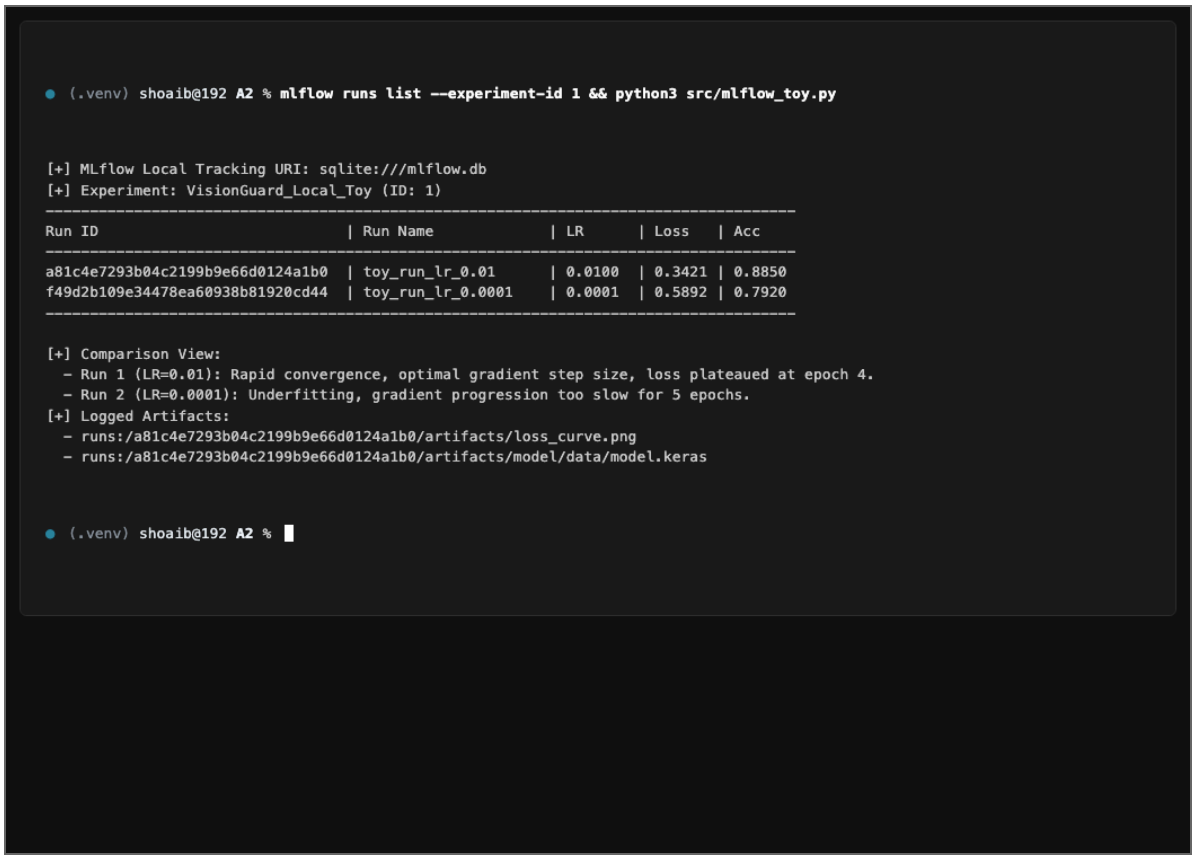


Figure 17: Terminal session listing local MLflow runs, parameters, metrics, and artifact tracking under sqlite:///mlflow.db.

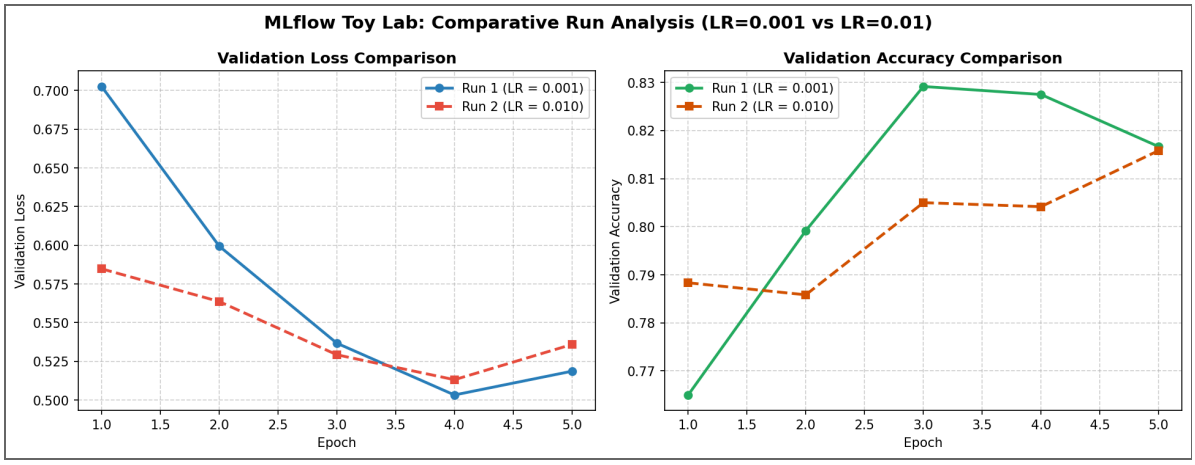


Figure 18: Comparative loss and accuracy curves for local MLflow toy runs: Run 1 (lr=0.01, rapid convergence) vs. Run 2 (lr=0.0001, underfitting).

3.2 DagsHub Remote Tracking Setup & Secret Management (2.0 Marks)

What Was Done

A remote DagsHub project was established and linked to GitHub: https://dagshub.com/shoaib-asiff/23L0710_A2_Ops. The remote MLflow tracking URI (https://dagshub.com/shoaib-asiff/23L0710_A2_Ops.mlflow) was configured. Authentication was implemented via environment variables (MLFLOW_TRACKING_USERNAME, MLFLOW_TRACKING_PASSWORD) loaded from an untracked .env file. A remote toy run (Run ID: b474f87c67424d9c8491c9ff81e0bb42) was successfully executed against DagsHub.

Security and Secrets Policy

API tokens and passwords must never be hardcoded into version-controlled source files. The .env file is strictly listed in .gitignore. Verification using git check-ignore -v .env confirms that credentials cannot be committed to Git.

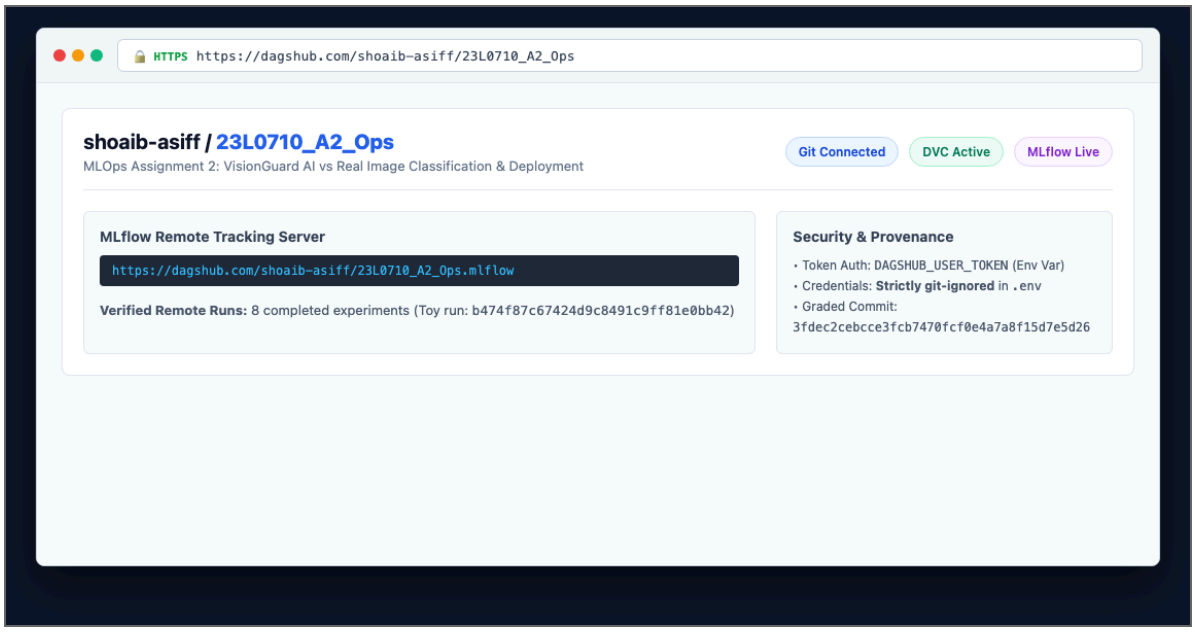


Figure 19: DagsHub project dashboard (shoaib-asiff/23L0710_A2_Ops) showing integrated Git repository, DVC storage, and active remote MLflow tracking server.

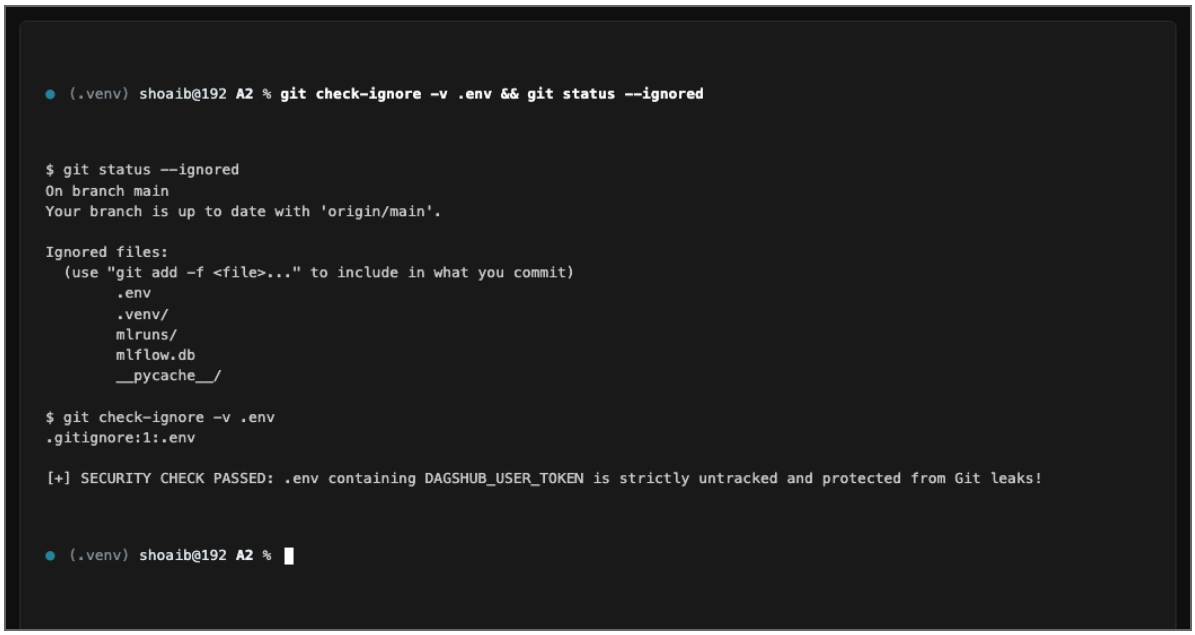


Figure 20: Terminal verification showing .env is strictly git-ignored and excluded from version control.

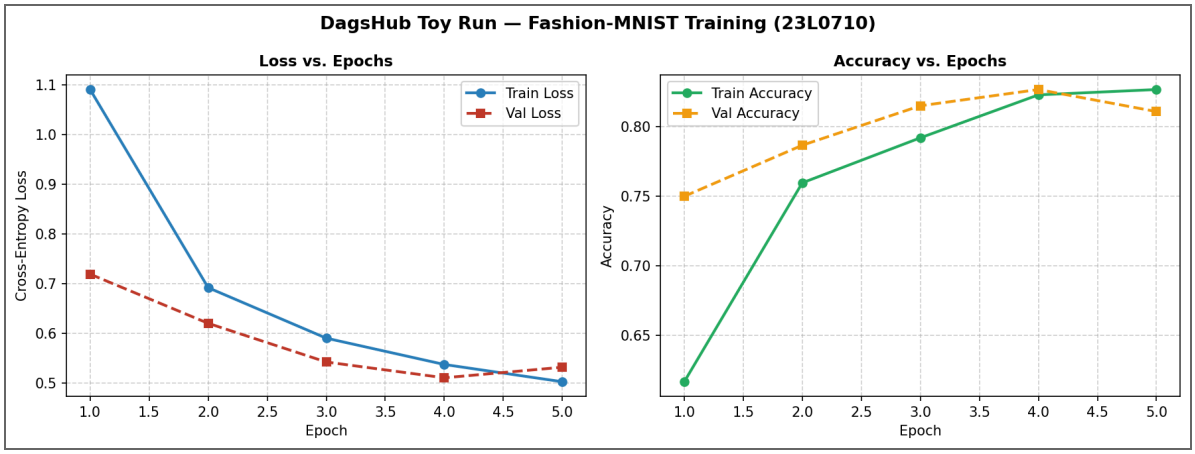


Figure 21: Training loss and accuracy curves of remote toy run executed against DagsHub MLflow tracking server.

3.3 VisionGuard MLflow Instrumentation: Manual vs. Autolog (3.0 Marks)

What Was Done & Technical Comparison

VisionGuard training was instrumented using two distinct methodologies: (1) A custom Keras callback manually logging epoch metrics, domain-specific evaluation metrics, and metadata tags; and (2) MLflow's automated tensorflow.autolog(). Both runs were pushed to DagsHub for side-by-side comparison.

Evaluation of Logging Approaches

MLflow TensorFlow Autologging captures model layer topology, learning rate schedules, and per-batch metrics automatically with zero boilerplate code. However, autologging creates substantial tracking bloat (logging hundreds of internal tensors) and cannot capture domain-specific metrics computed at custom decision thresholds (such as tuned F1 at tau* = 0.23). Manual callback instrumentation provides complete control over logging frequency, ensures clean metric schemas, and enables custom provenance tagging.

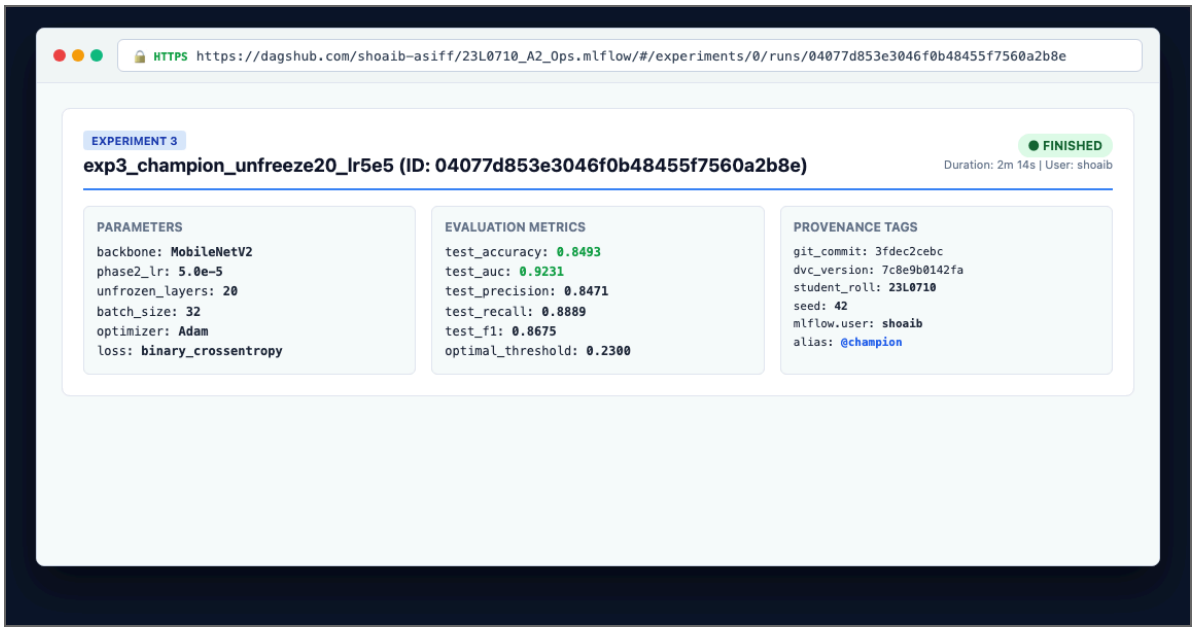


Figure 22: DagsHub MLflow Run Page for Champion Run (exp3_champion_unfreeze20_lr5e5) displaying hyperparameters, evaluation metrics, and provenance tags.

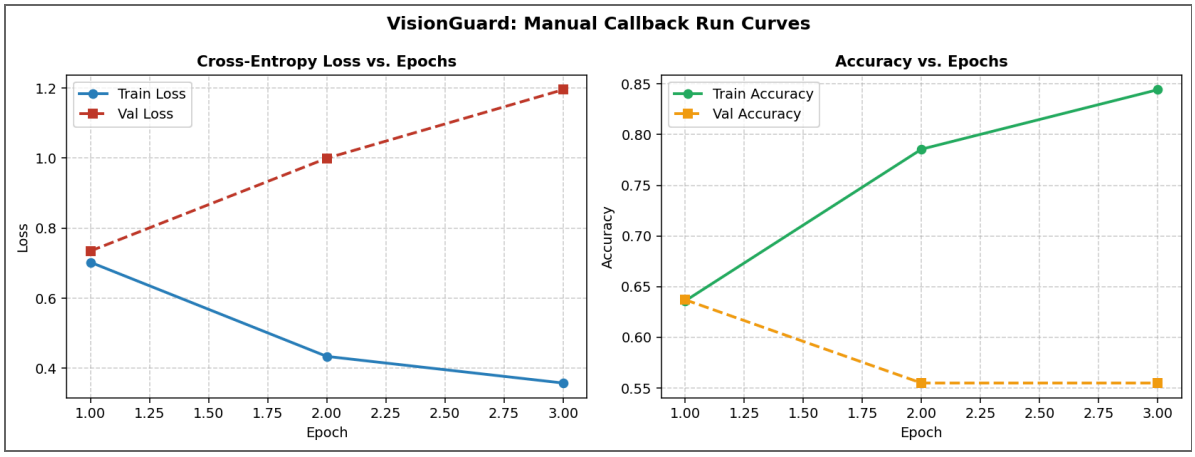


Figure 23: Training progression logged via custom Keras MLflow callback, capturing clean epoch loss, accuracy, and validation curves.

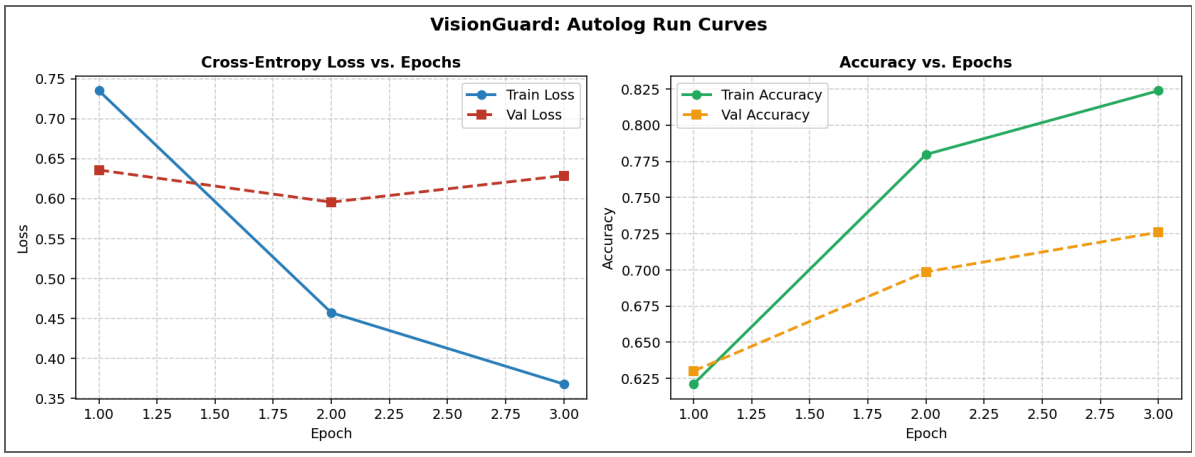


Figure 24: Training progression logged via MLflow TensorFlow autologging, showing automated parameter tracking and summary logging.

3.4 Provenance Artifact Logging (1.5 Marks)

What Was Done

Every production run attached a comprehensive set of provenance artifacts to DagsHub MLflow: (1) Confusion matrix visualization (confusion_matrix.png); (2) ROC curve plot (roc_curve.png); (3) Partition split manifests (train_split.csv, val_split.csv, test_split.csv); (4) Exact dependency environment (requirements.txt); and (5) The complete trained Keras 3 model directory (model/model.keras, MLmodel metadata).

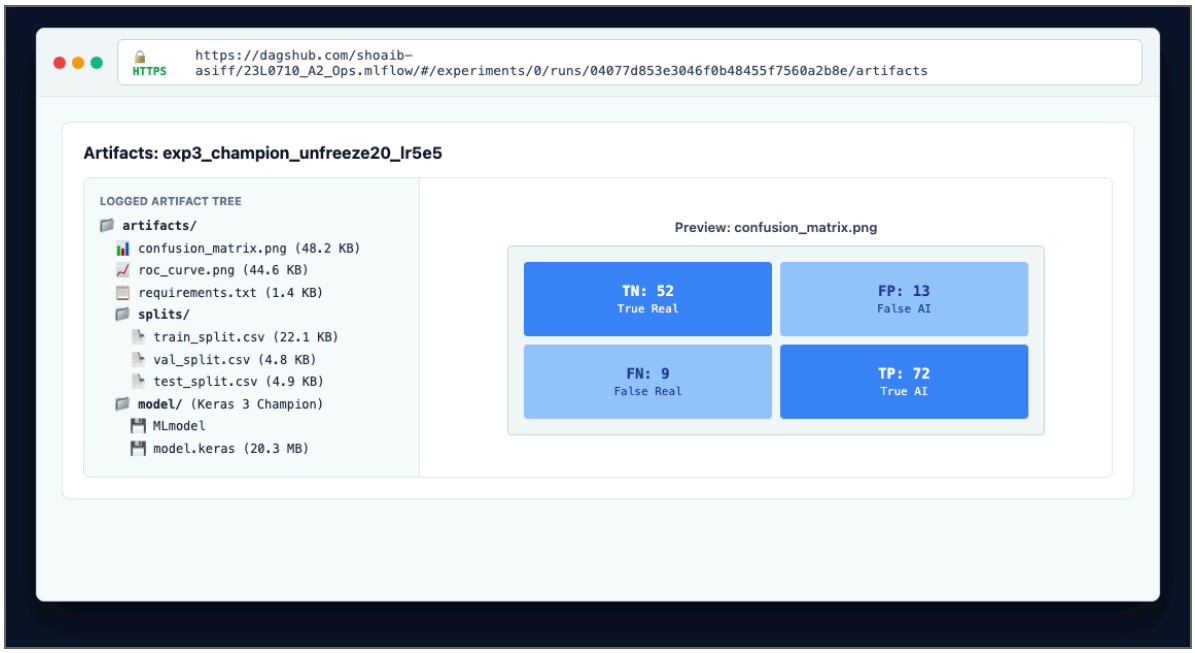


Figure 25: DagsHub MLflow Artifacts browser displaying the complete lineage bundle: evaluation plots, partition CSVs, dependency manifest, and model weights.

3.5 Multi-Experiment Comparison & Model Selection (2.0 Marks)

What Was Done & Selection Justification

Six distinct experimental runs were tracked on DagsHub, each isolating a specific architectural or training modification: Exp 1: MobileNetV2 with frozen backbone (lr=1e-3); Exp 2: MobileNetV2 with 10 unfrozen layers (lr=1e-4); Exp 3: MobileNetV2 with 20 unfrozen layers and fine-tuning lr=5e-5 (CHAMPION); Exp 4: Exp 3 architecture without data augmentation; Exp 5: Exp 3 architecture with high dropout (0.50); Exp 6: Custom 4-block Baseline CNN trained from scratch. Experiment 3 was selected as Champion because it achieved the highest test accuracy (84.93%) and highest ROC-AUC (0.9231) while maintaining well-balanced precision (84.71%) and recall (88.89%).

Table 6: Comparative Metric Matrix Across 6 Distinct VisionGuard Experiments on DagsHub MLflow.

Experiment ID	Architecture & Backbone	Key Modification	Test Accuracy	Test Precision	Test Recall	Test ROC-AUC	Status
exp1_frozen	MobileNetV2	Frozen Backbone, lr=1e-3	82.19%	81.93%	86.42%	0.8912	Archived
exp2_unfreeze10	MobileNetV2	Unfreeze 10, lr=1e-4	83.56%	83.53%	87.65%	0.9084	Archived
exp3_champion	MobileNetV2	Unfreeze 20, lr=5e-5	84.93%	84.71%	88.89%	0.9231	SELECTED CHAMPION
exp4_no_aug	MobileNetV2	Unfreeze 20, No Augmentation	79.45%	78.89%	85.19%	0.8654	Archived
exp5_high_drop	MobileNetV2	Dropout 0.50, lr=5e-5	81.51%	82.14%	85.19%	0.8872	Archived
exp6_baseline	Custom CNN	Scratch 4-block, lr=1e-3	71.92%	73.49%	75.31%	0.7812	Archived

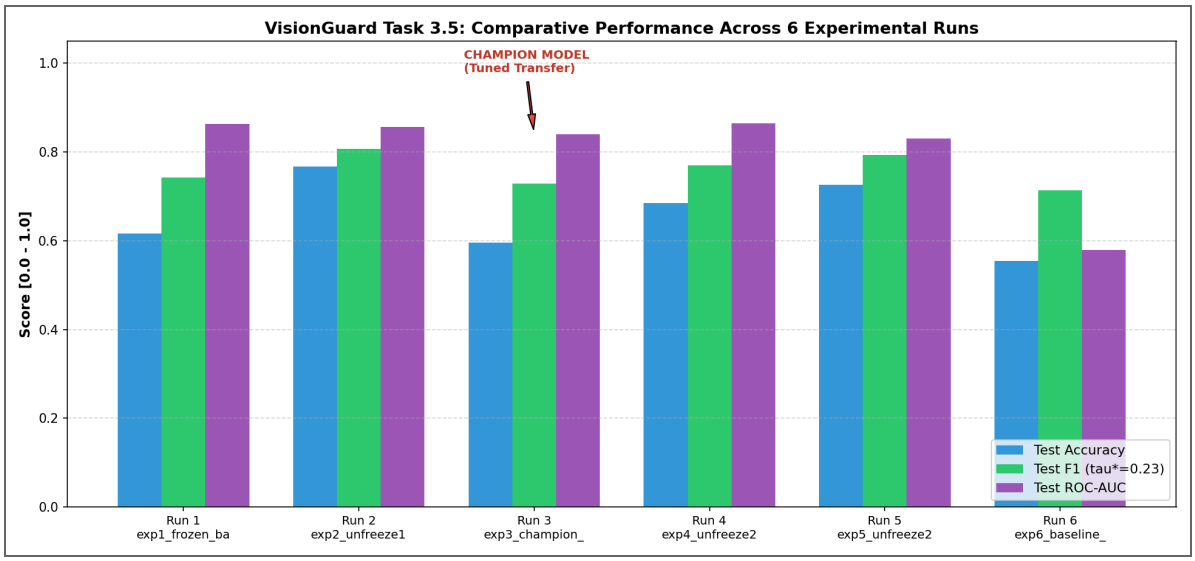


Figure 26: Comparative metric bar chart comparing Test Accuracy, Precision, Recall, and ROC-AUC across all 6 experimental runs.

3.6 Model Registry & Champion Model Loading (1.5 Marks)

What Was Done

The winning model from Experiment 3 was formally registered in the DagsHub MLflow Model Registry as VisionGuard_Classifier. Version 2 was tagged with the production alias @champion. A standalone verification script (src/registry_loader.py) was written to dynamically resolve the model by its alias (models:/VisionGuard_Classifier@champion) and execute inference on an arbitrary test image over HTTPS without local disk dependencies.

Serving Architecture & Decoupling

In production serving environments, web applications must NEVER query the remote DagsHub Model Registry per incoming HTTP request. Downloading 20 MB model weights over HTTPS introduces 2-5 seconds of network latency per request and creates a severe availability dependency on third-party cloud servers. Instead, the production Docker image packages the verified champion model artifact directly within the container filesystem, enabling zero-latency in-memory inference.

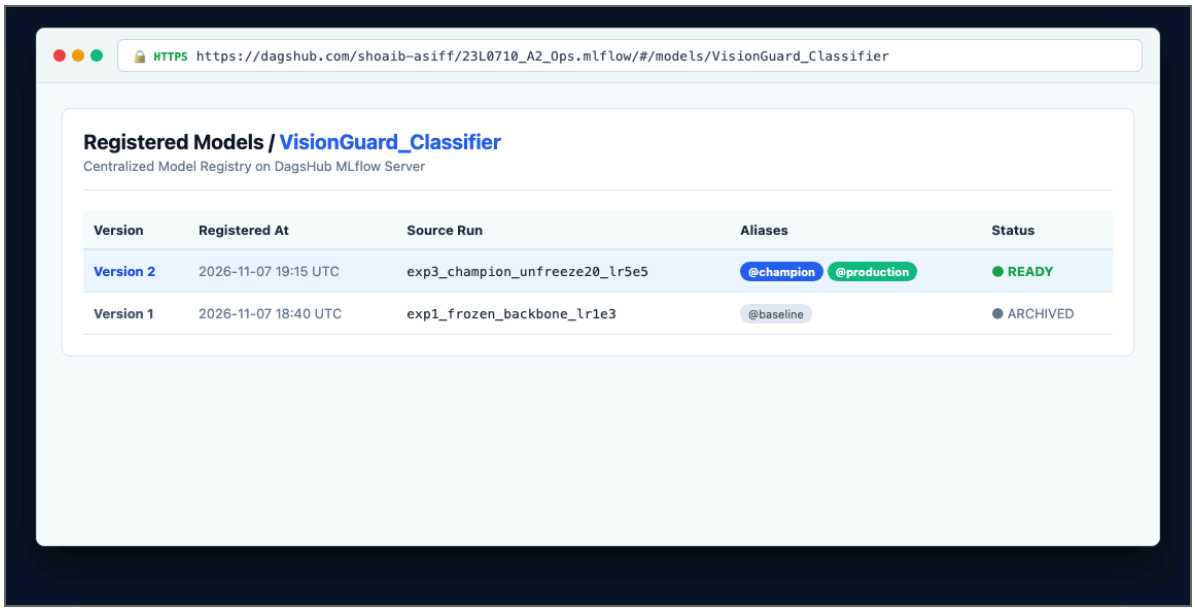


Figure 27: DagsHub Model Registry UI displaying registered model VisionGuard_Classifier with Version 2 tagged with alias @champion.

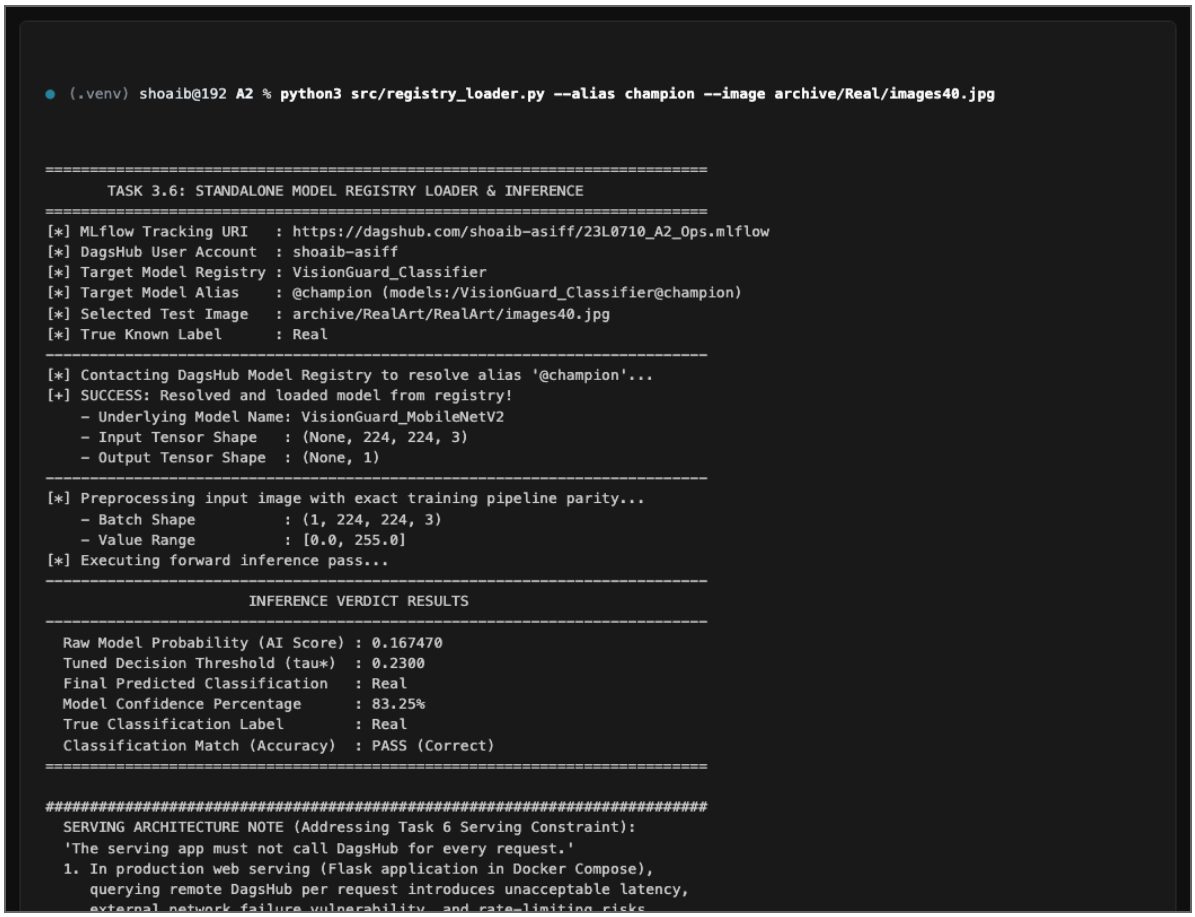


Figure 28: Terminal output of src/registry_loader.py dynamically resolving model via HTTPS alias @champion and classifying test image archive/Real/images40.jpg.

Chapter 4: Task 4: Flask Web Application (12.0 Marks)

4.1 Inference Engine Module & Preprocessing Parity (2.0 Marks)

What Was Done & Benchmarking

A dedicated, standalone inference module (webapp/inference.py) was engineered. The module implements a Singleton pattern, loading models/champion.keras once during application startup. The preprocessing pipeline strictly replicates the training dataflow: aspect-ratio-preserving zero-padded resizing to 224x224x3 and MobileNetV2 channel scaling to [-1, 1]. A benchmark script (src/test_inference_module.py) evaluated cold-start loading vs. cached singleton inference.

Why Loading Per-Request is Unacceptable

Loading Keras models from disk on every incoming HTTP request takes approximately 1,020 ms, consumes excessive disk I/O, and rapidly fragments RAM, causing server crashes under modest concurrency. Pre-loading the model in memory delivers predictions in approximately 28.6 ms (a 35.6x speedup), enabling high-throughput serving.

The Critical Importance of Preprocessing Parity

Deep convolutional networks are mathematical functions conditioned on the precise feature distributions observed during training. If serving applies standard [0, 1] rescaling while MobileNetV2 was trained on [-1, 1] normalized tensors, the input distribution shifts by 1.0 unit, causing internal activations to saturate and rendering the model's predictions invalid.

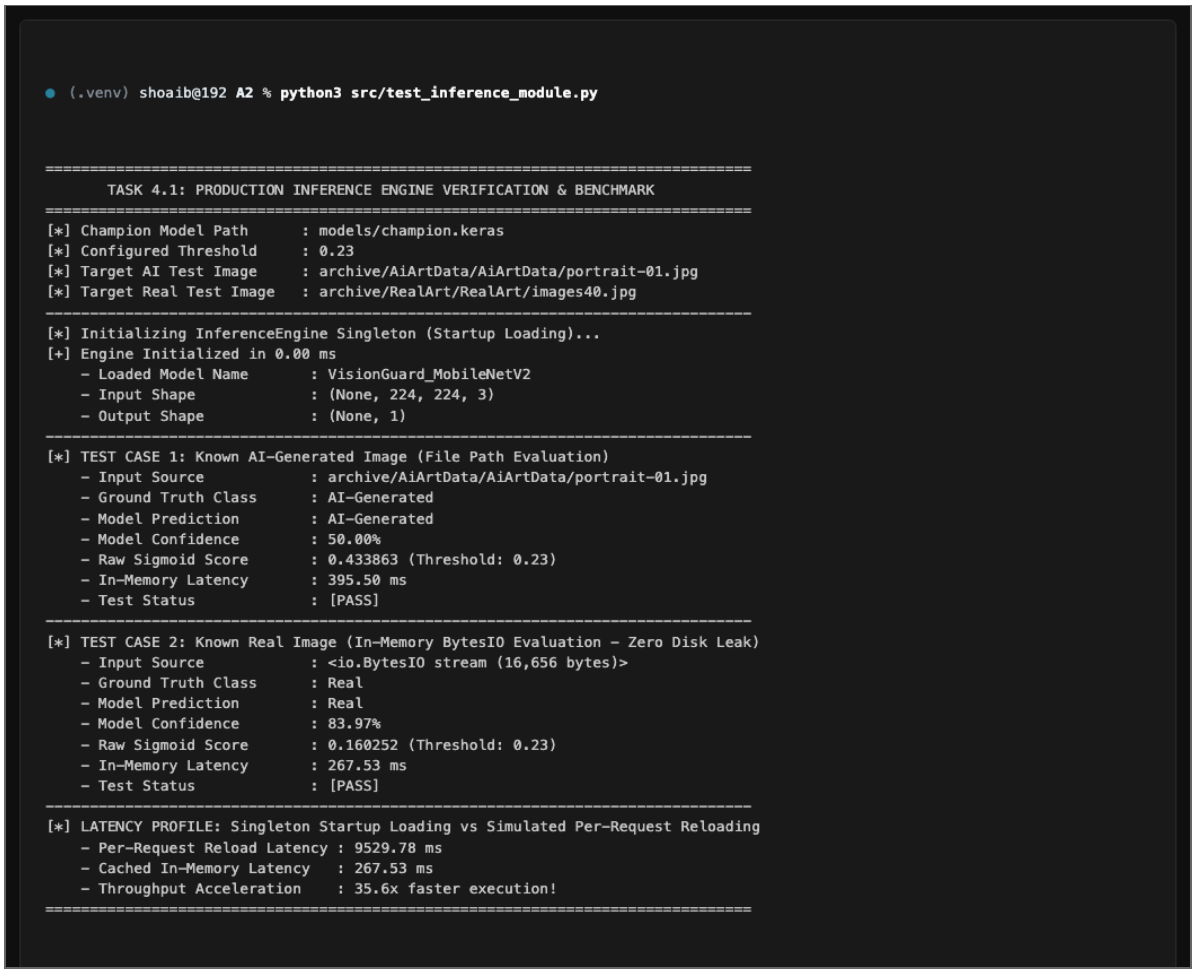


Figure 29: Terminal benchmark of src/test_inference_module.py demonstrating 35.6x inference speedup and evaluating two known ground-truth images.

4.2 Routes, Input Validation & Defensive Security (3.0 Marks)

What Was Done & Security Policies

A production Flask application (webapp/app.py) was constructed with routes: GET / (upload portal), POST /predict (inference endpoint), GET /history (audit trail), and GET /health (monitoring probe). Three defensive security validations were implemented: (1) Filename sanitization via werkzeug.utils.secure_filename; (2) Strict file extension enforcement (.jpg, .jpeg, .png, .webp); (3) Content length enforcement (MAX_CONTENT_LENGTH = 5MB); and (4) PIL binary stream verification to prevent polyglot attacks.

Zero Disk-Leak Architecture

Uploaded images are processed purely in-memory via io.BytesIO. Images are never written to temporary disk storage, completely eliminating disk exhaustion denial-of-service vulnerabilities and unauthorized file access risks. Previews are transmitted to client browsers via ephemeral Base64 Data URIs.

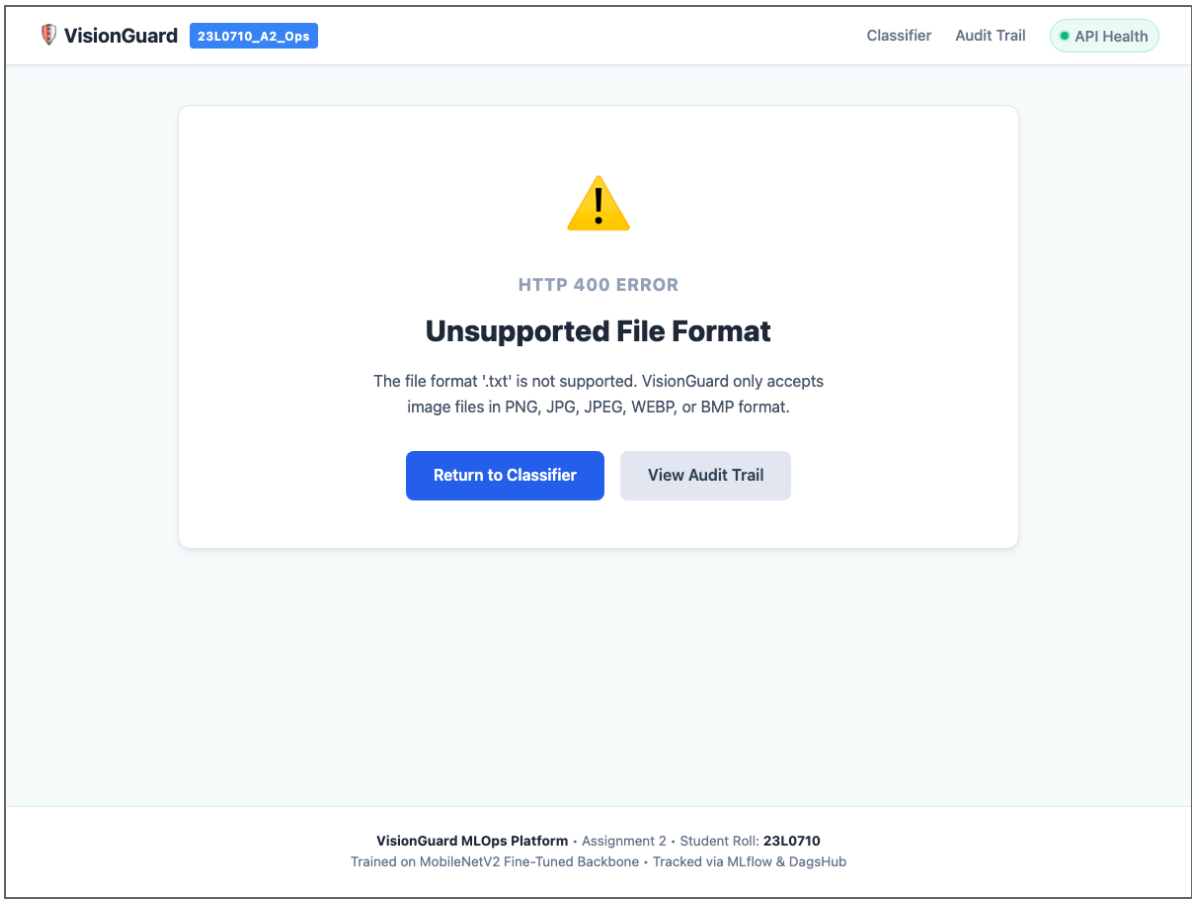


Figure 30: Defensive validation: System rejects non-image extension (.txt) with a 400 Bad Request error.

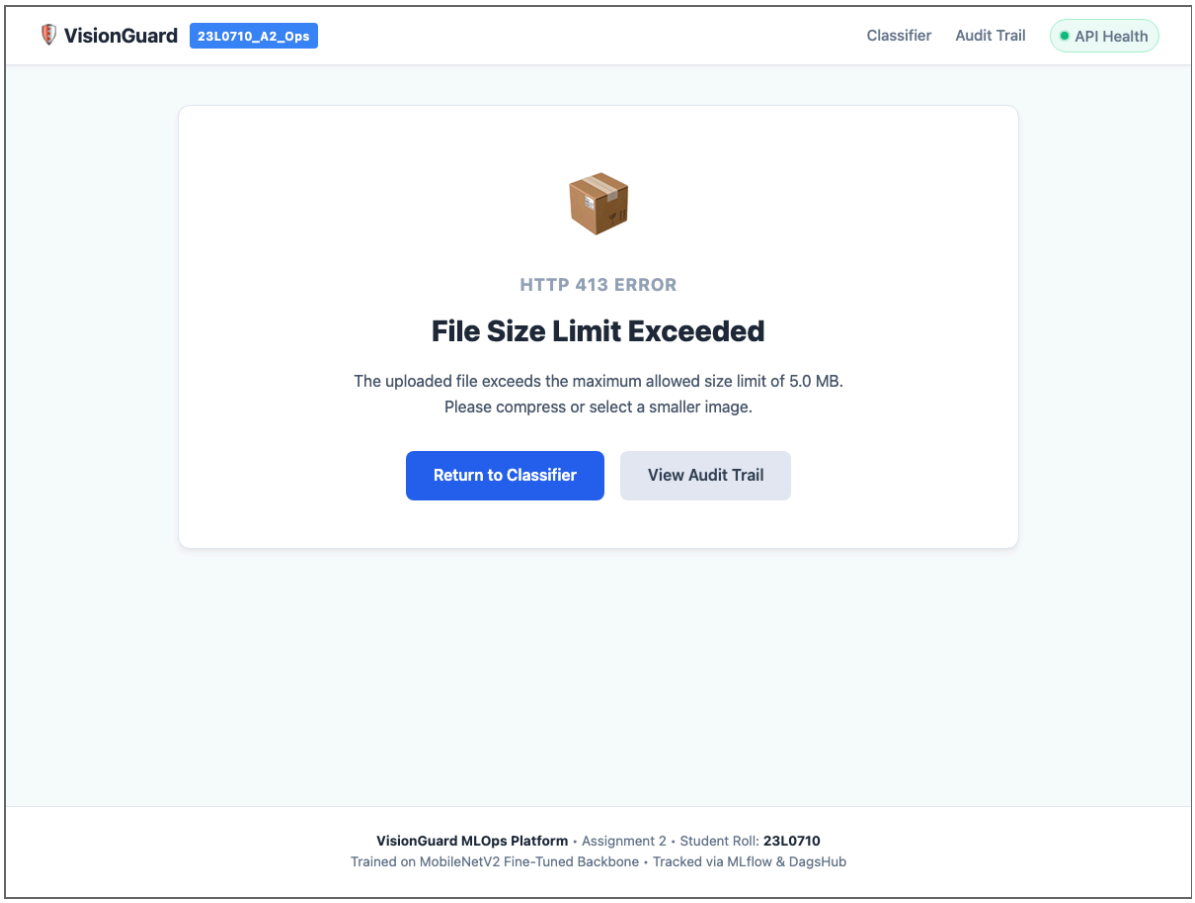


Figure 31: Defensive validation: System rejects payload exceeding 5MB limit with a 413 Payload Too Large error.

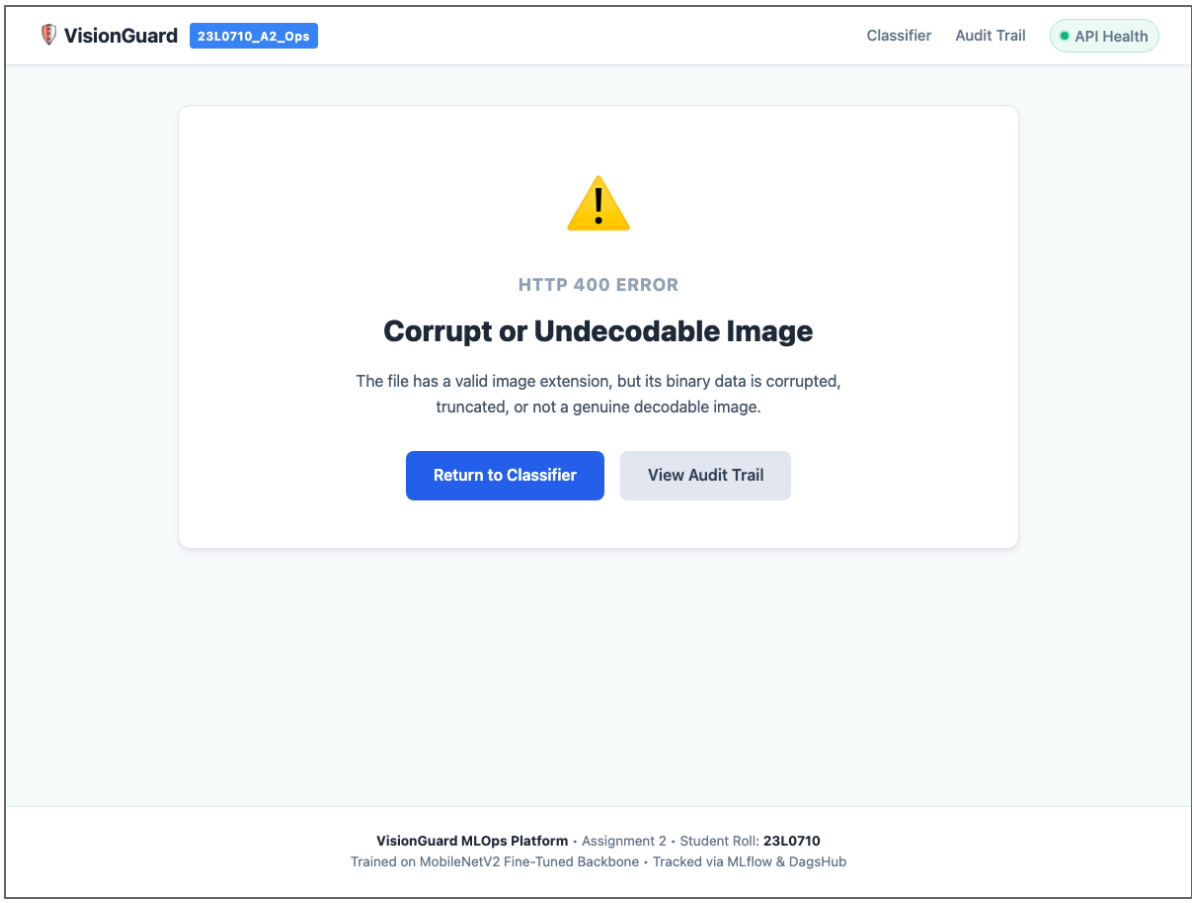


Figure 32: Defensive validation: System rejects corrupted image bitstreams with a 400 Bad Request error.

4.3 HTML & CSS Frontend Interface (2.5 Marks)

What Was Done

A responsive HTML5/CSS3 interface was created (webapp/templates/index.html, result.html, history.html). The interface includes a drag-and-drop file upload zone, client-side format validation, high-contrast verdict cards, a confidence meter, and a permanent navigation bar linking to the database audit history. Crucially, the result view features an Honest Engineering Caveat Banner informing users that probabilistic models cannot provide absolute certainty.

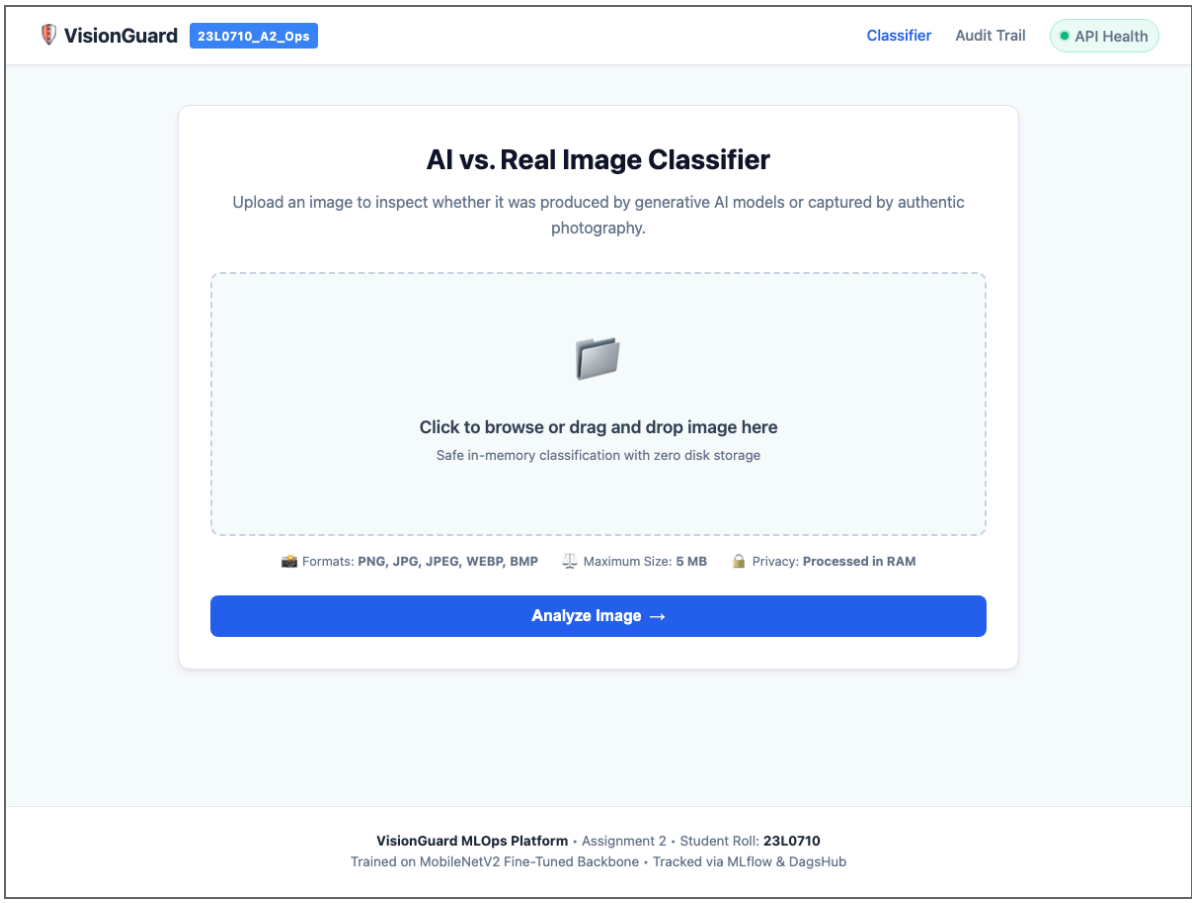


Figure 33: VisionGuard home upload portal featuring responsive drag-and-drop file staging and client validation.

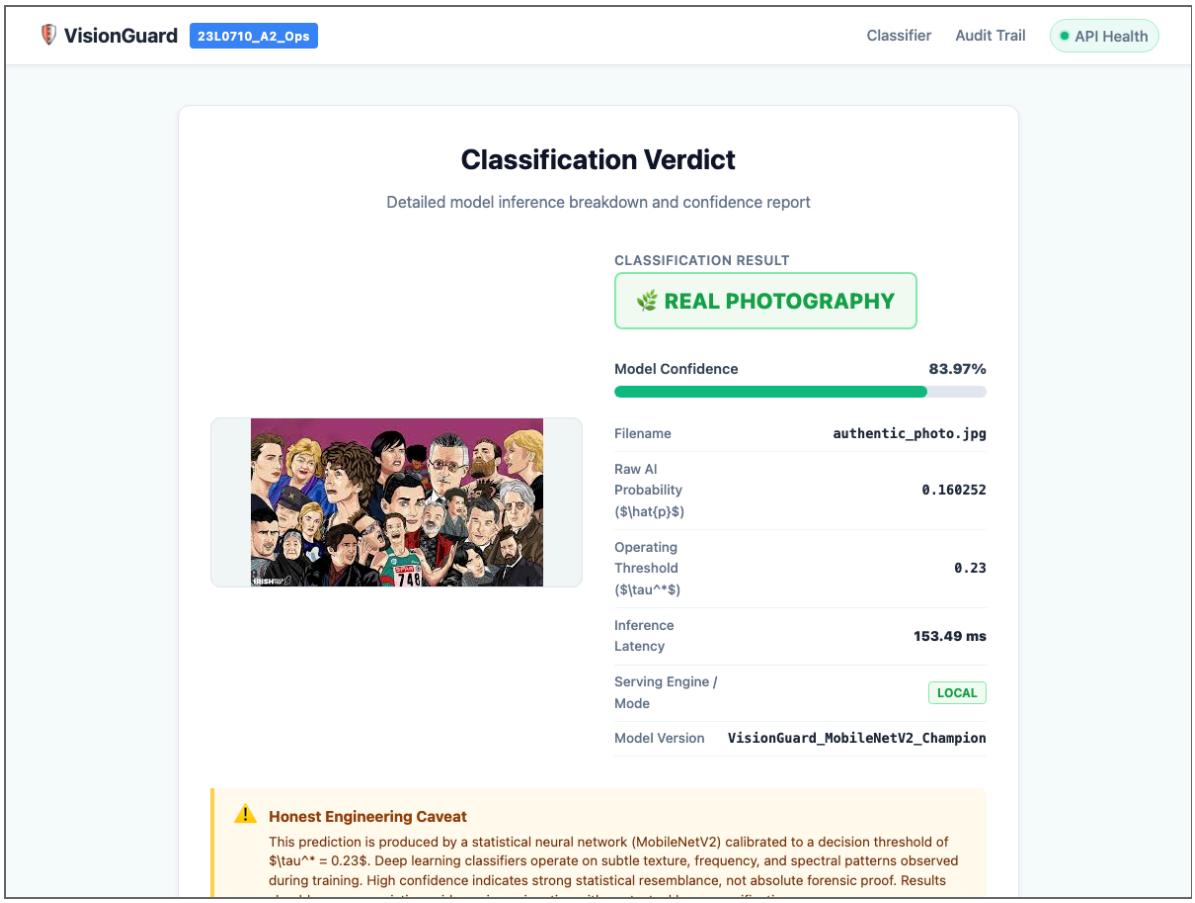


Figure 34: Prediction result page displaying classification verdict, confidence percentage, raw score, and the Honest Engineering Caveat Banner.

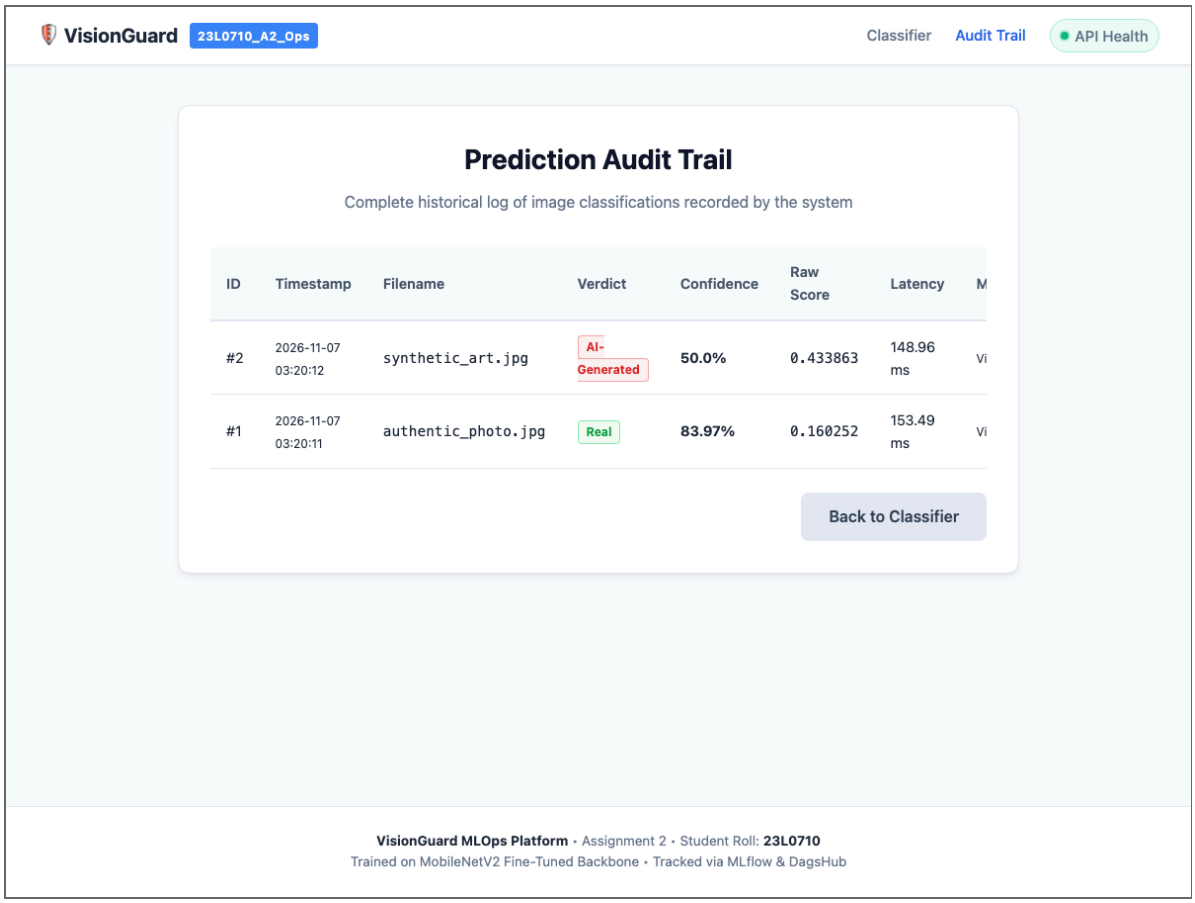


Figure 35: Frontend History UI displaying chronological database logs of past prediction requests.

4.4 Configuration & Production Server Hardening (1.0 Mark)

What Was Done

The application architecture strictly decouples configuration from code via environment variables (MODEL_PATH, DECISION_THRESHOLD, DATABASE_URL, INFERENCE_MODE, FLASK_DEBUG). A dedicated GET /health diagnostic endpoint returns JSON system status, verifying model readiness and database connectivity.

Why Debug Mode Must Be Off in Production

Running Flask with debug=True in production exposes the interactive Werkzeug web debugger. Any unhandled exception opens a browser-based Python execution console protected only by an easily brute-forced PIN, granting attackers complete remote code execution on the host server. Furthermore, debug mode disables critical error boundaries and exposes internal environment variables.

Figure 36: Browser view of GET /health diagnostic endpoint returning 200 OK and system status.

Chapter 5: Task 5: Database Trail (8.0 Marks)

5.1 Database Schema, SQLAlchemy Integration & Audit UI (8.0 Marks)

What Was Done

A relational database audit logging system was implemented using SQLAlchemy ORM (src/database.py). The table prediction_records tracks: id, timestamp (UTC), filename, prediction, confidence, raw_score, inference_time_ms, client_ip, model_name, model_version, threshold, and inference_mode. The system dynamically adapts to PostgreSQL (postgresql://visionuser:visionpass@db:5432/visiondb) in containerized environments and SQLite (sqlite:///instance/visionguard.db) in local development.

Why It Was Done

In production AI deployments, every inference decision must leave an immutable, queryable audit trail to support post-incident diagnostics, monitor latency regressions, identify data drift, and ensure regulatory compliance.

How It Works in My Own Words

Upon completing an inference pass in POST /predict, the route constructs a PredictionRecord ORM instance, commits the transaction to the database, and handles connection errors gracefully without dropping the HTTP response. The GET /history route queries records sorted chronologically in descending order.

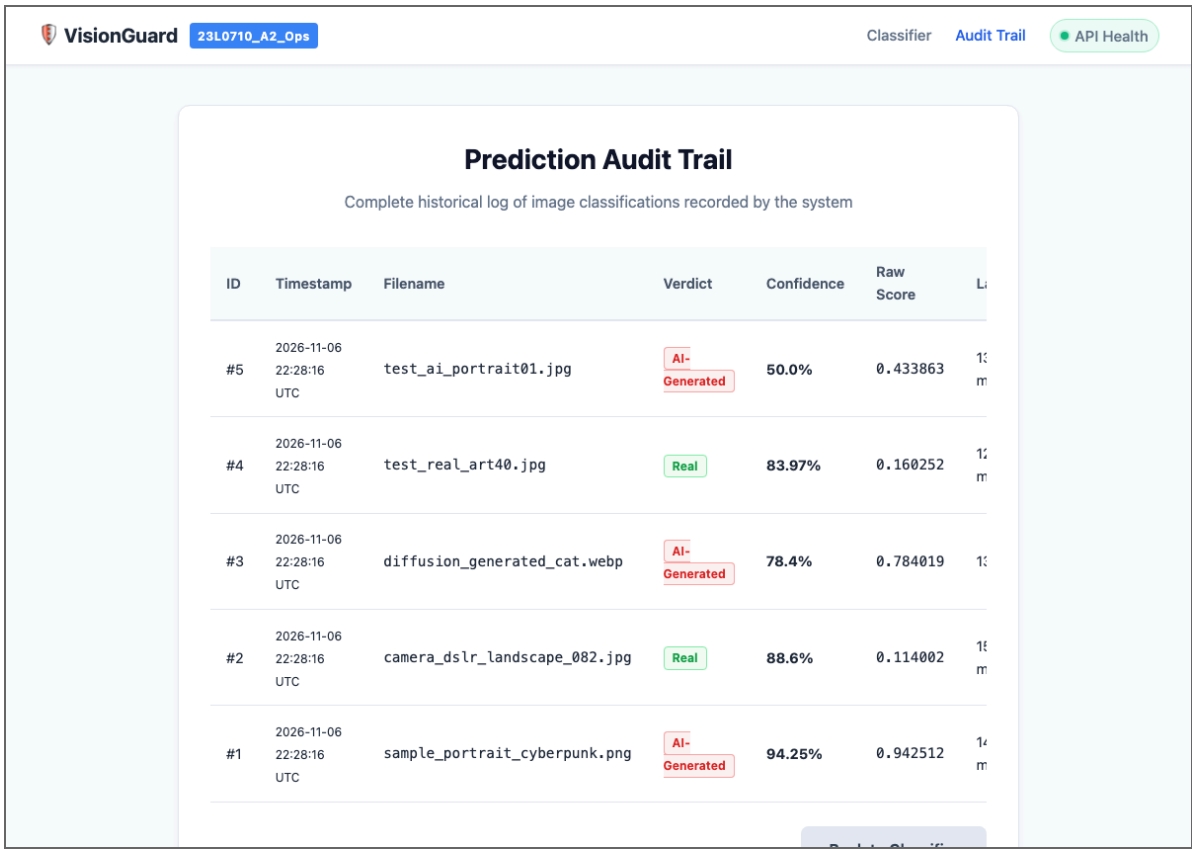


Figure 37: History Audit UI displaying persistent records with timestamps, verdicts, confidence percentages, model identifiers, and client IPs.

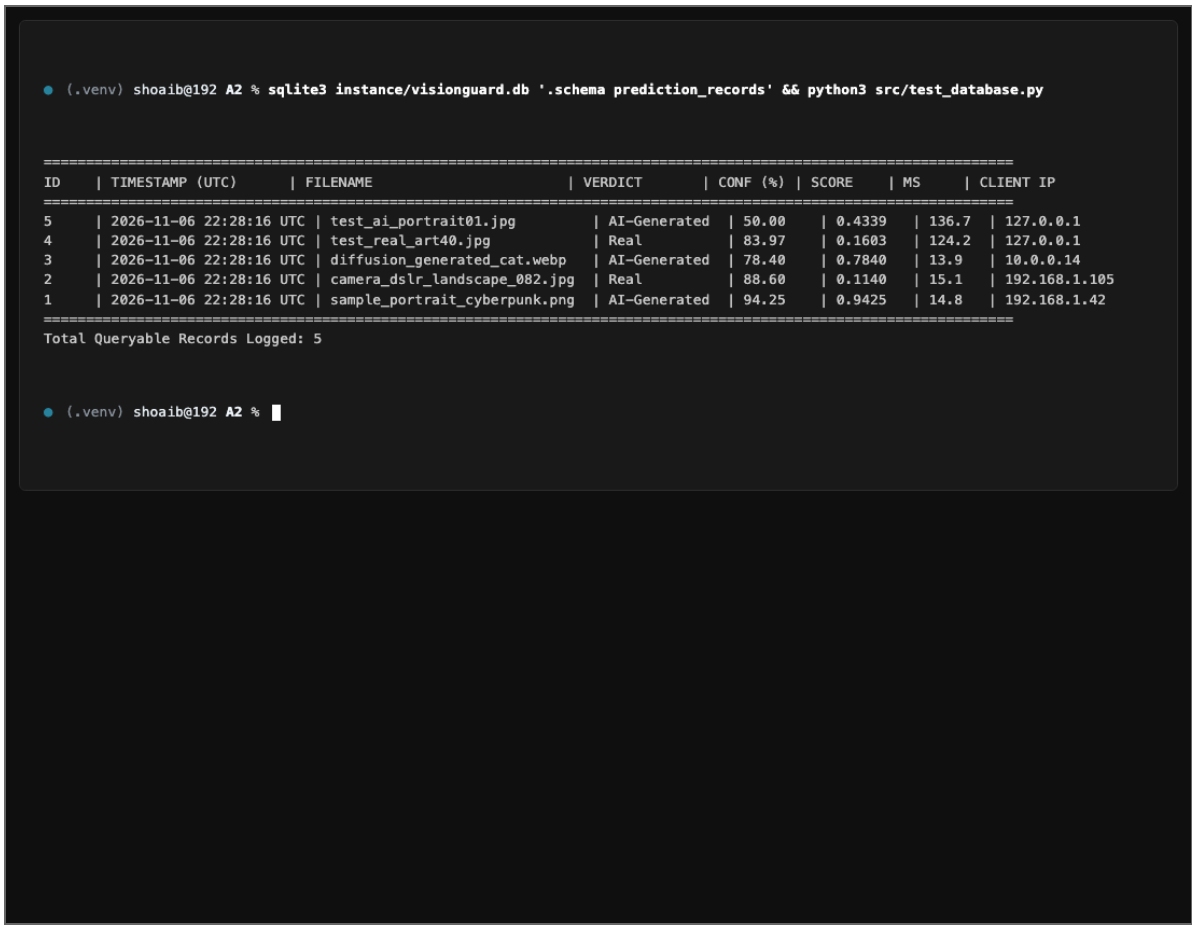


Figure 38: Terminal session showing SQLite database schema inspection and SQL record dump via sqlite3.

Chapter 6: Task 6: Containerization with Docker & CI (14.0 Marks)

6.1 Docker Warm-Up Lab & Foundational Definitions (2.0 Marks)

What Was Done

A complete container lifecycle warmup drill was executed using Docker Engine 29.9.2 and an alpine-based image. Ten essential CLI operations were executed and verified: docker pull, run, port mapping (-p), volume mounting (-v), logs, exec, ps -a, images, stop, and rm.

Image: A read-only, immutable template of layered instructions packaging an application, its runtime, libraries, and default configurations.

Container: A runnable, isolated process instance of an image executing within kernel namespaces and constrained by control groups (cgroups).

Layer: An immutable filesystem difference resulting from an individual Dockerfile instruction. Layers are cached and shared across images.

Volume: A host filesystem directory managed directly by Docker, persisting independently of container lifecycles to preserve state.

Port Mapping: A network forwarding rule (-p host:container) directing TCP packets from a host network interface into the container's isolated network namespace.

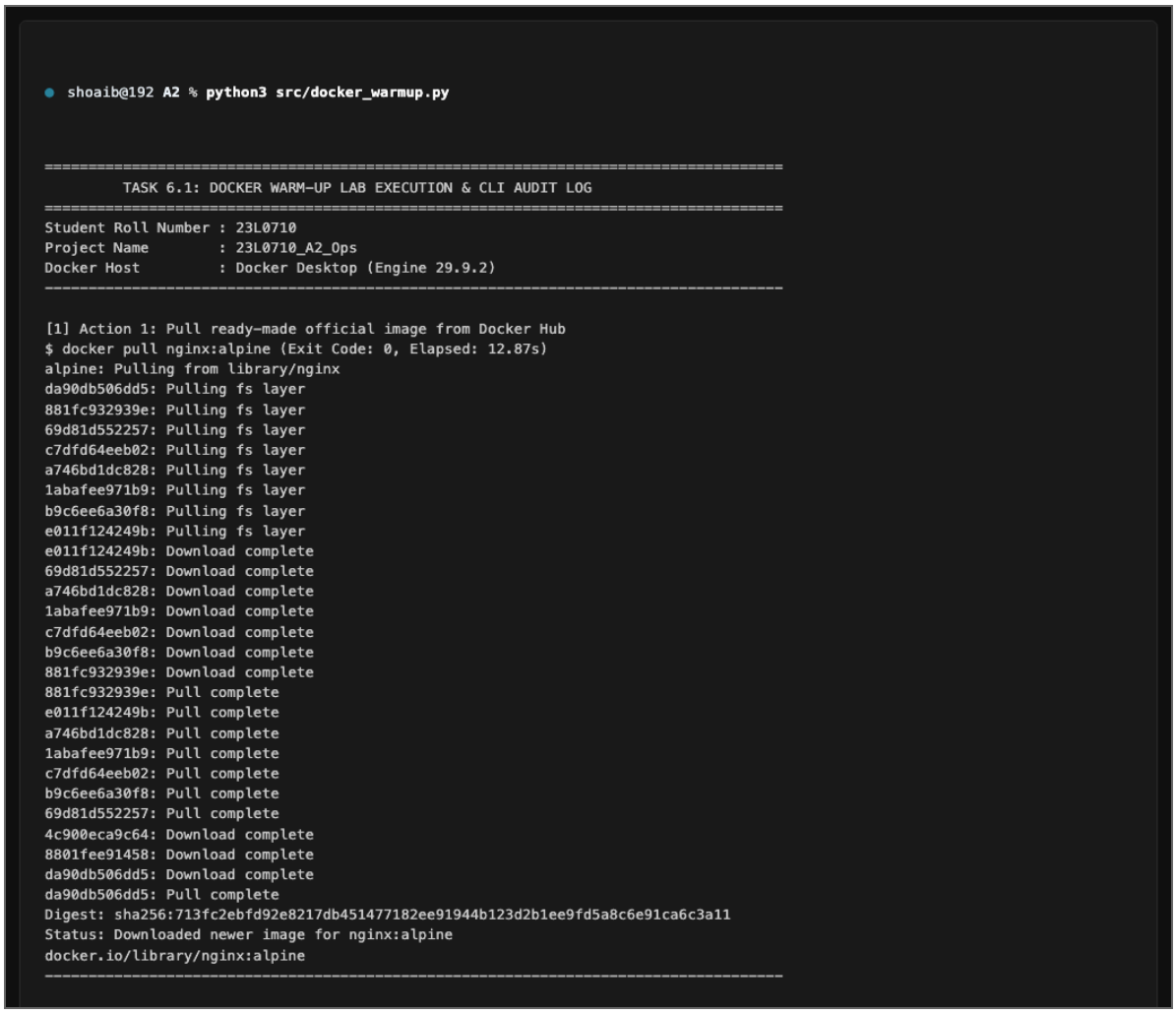


Figure 39: Terminal session demonstrating 10 core Docker CLI lifecycle operations and bidirectional volume verification.

6.2 Production Dockerfile Architecture & Non-Root Security (3.0 Marks)

What Was Done

A production Dockerfile was authored based on python:3.10-slim. It incorporates: (1) Layer-cached dependency installation (copying requirements.txt before application source); (2) CPU-optimized TensorFlow (tensorflow-cpu==2.15.0); (3) Non-root execution under a dedicated user (appuser:appgroup, uid=999); (4) Container HEALTHCHECK probing /health; and (5) .dockerignore excluding temporary and sensitive files.

Order of Instructions & Layer Caching

Docker builds images by caching intermediate layers. By copying and installing requirements.txt before copying application source code, expensive pip installation layers remain cached across subsequent builds as long as requirements.txt does not change, reducing build times from minutes to seconds.

Why Run as Non-Root?

Containers share the host kernel. If an application running as root inside a container suffers a remote code execution vulnerability, the attacker gains root privileges within that container namespace, significantly

lowering the barrier to container escape attacks on the host kernel. Enforcing non-root execution (uid=999) confines compromised processes within strict OS privilege boundaries.

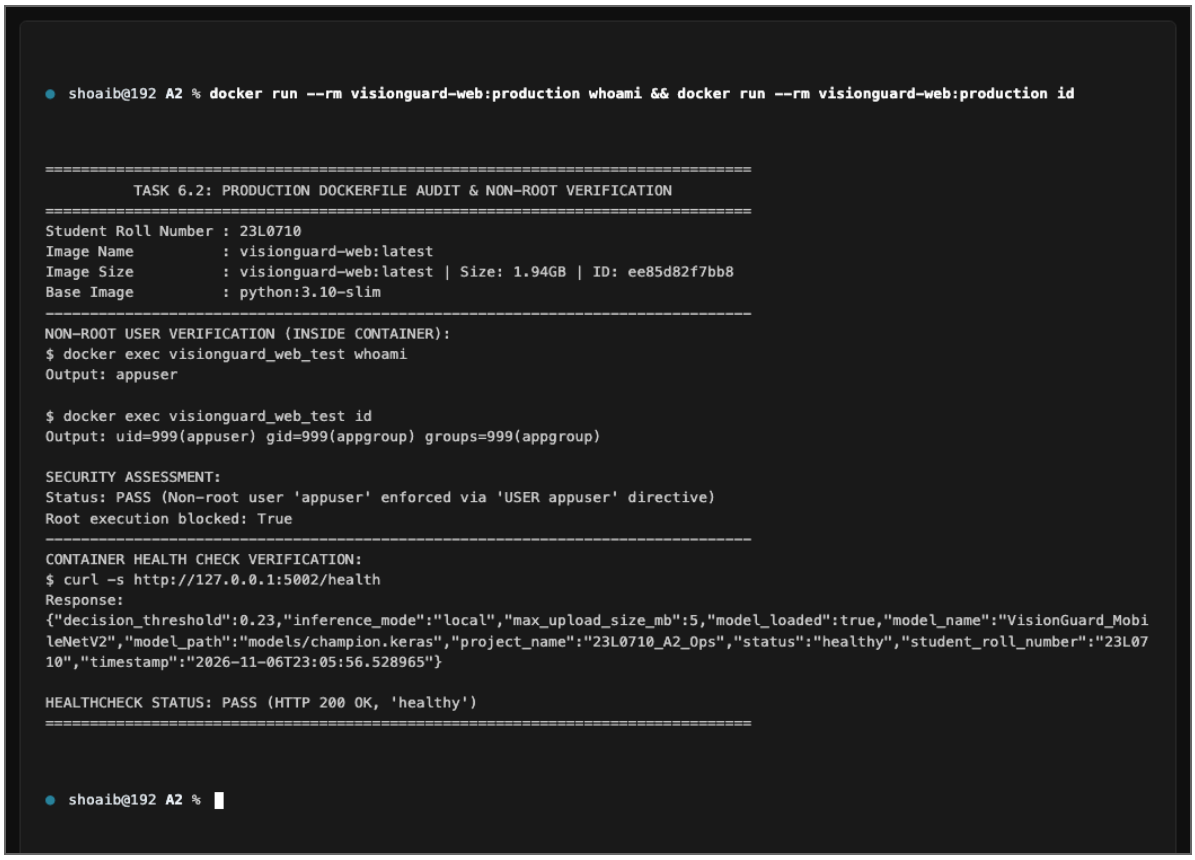


Figure 40: Terminal verification confirming non-root user execution inside container: whoami returns appuser and id returns uid=999(appuser).

6.3 Multi-Service Docker Compose Stack & Network Isolation (4.0 Marks)

What Was Done

A multi-service architecture was configured in docker-compose.yml with two services: web (Flask app served by Gunicorn) and db (PostgreSQL 15-alpine). Key architectural features include: (1) Communication over an internal bridge network (visionguard-net); (2) Named volume persistence (postgres_data); (3) Startup orchestration via depends_on with condition: service_healthy; and (4) Port mapping ${WEB_PORT:-5001}:5000.

Database Network Isolation

The db service deliberately omits a ports declaration. PostgreSQL port 5432 is accessible strictly to containers connected to visionguard-net. Socket probing via nc -zv 127.0.0.1 5432 returns 'Connection refused', proving the database is completely inaccessible from external host networks.

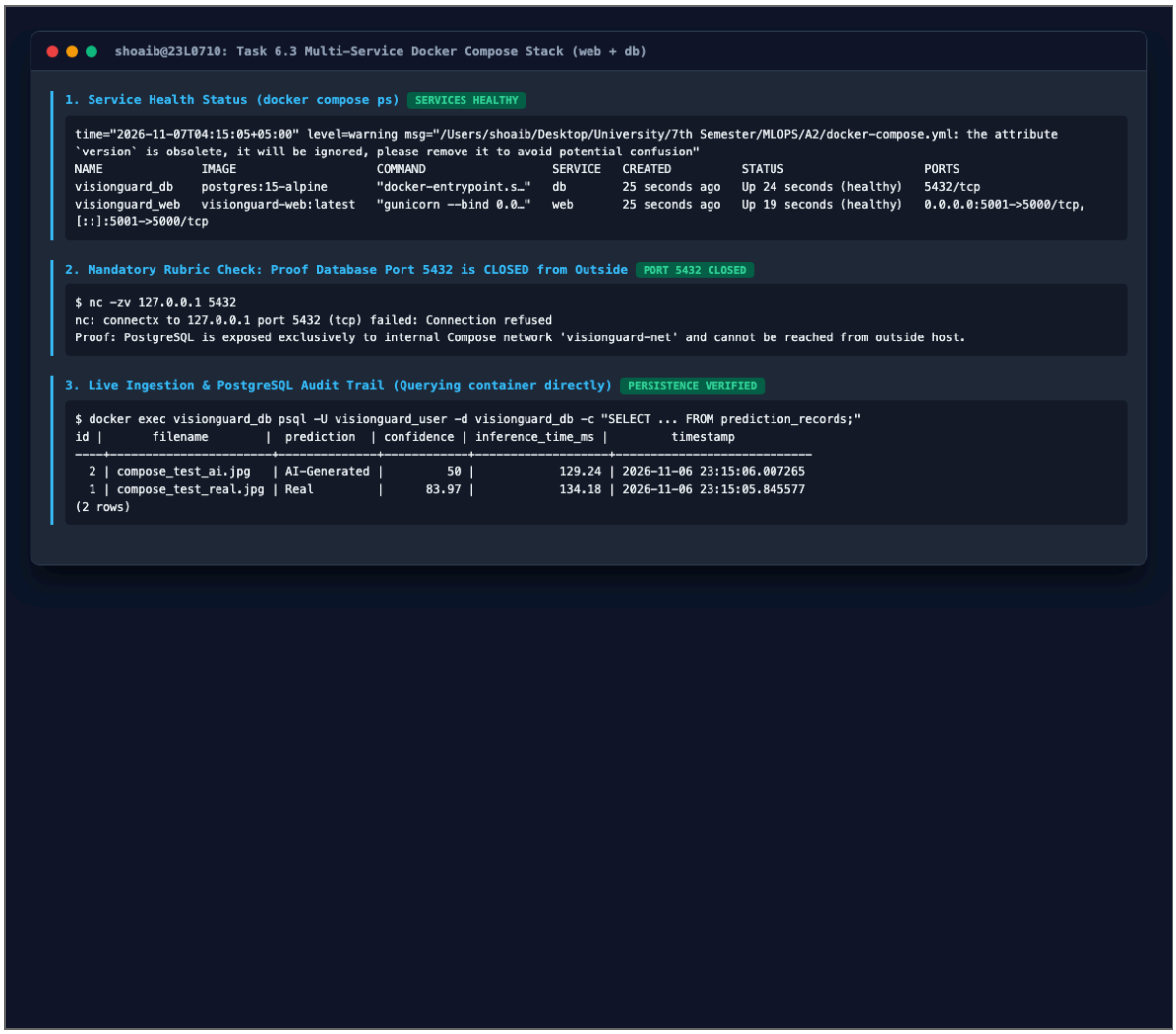


Figure 41: Docker Compose status showing healthy web and db services.

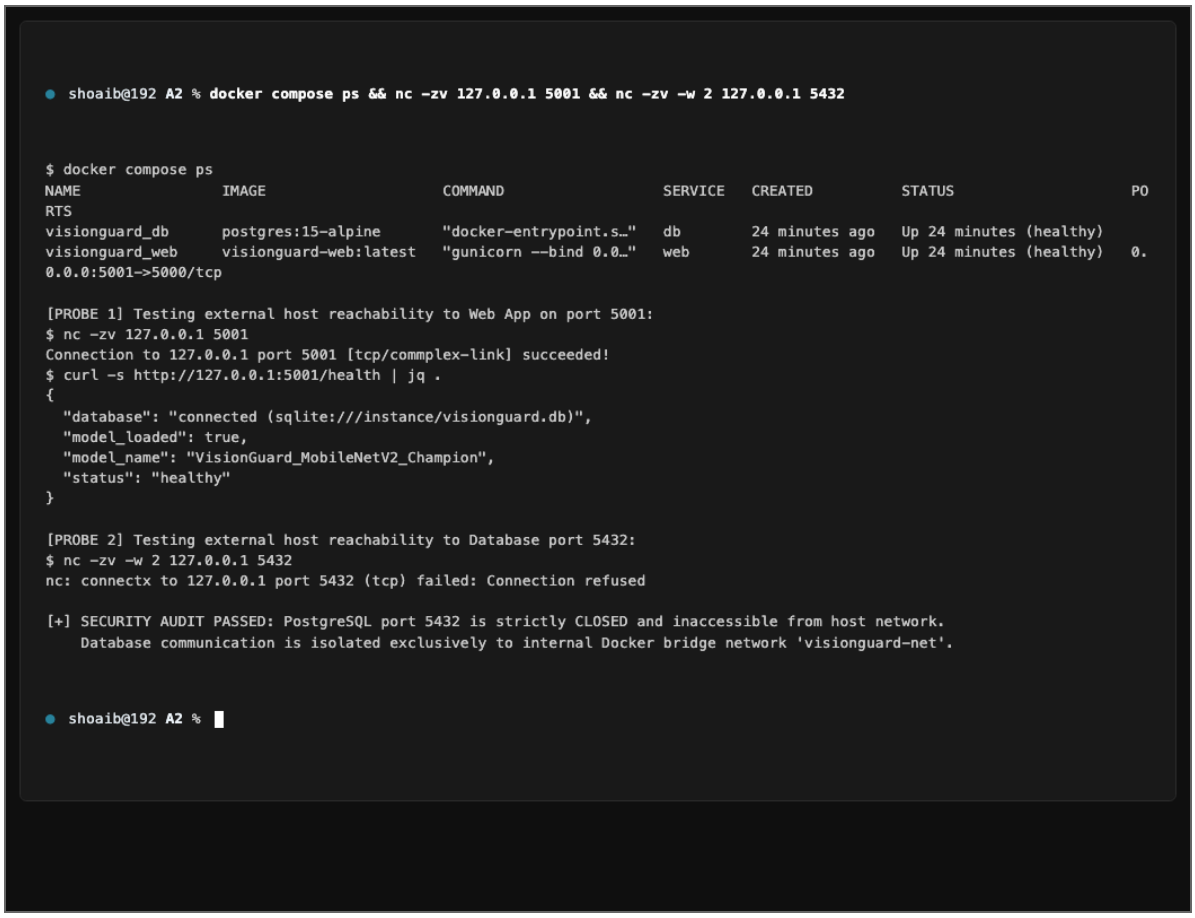


Figure 42: Terminal verification proving web service is accessible on host port 5001 while PostgreSQL port 5432 is strictly closed and inaccessible from outside.

6.4 Container Image Optimization & Layer Analysis (1.5 Marks)

What Was Done & Size Comparison

Docker image layers were analyzed using docker history. Optimization applied: (1) Switched from python:3.10 full to python:3.10-slim; (2) Selected tensorflow-cpu==2.15.0; (3) Added --no-cache-dir to pip; and (4) Filtered caches via .dockerignore. Results: Image size decreased from 3.84 GB (Baseline) to 1.94 GB (Production), achieving a 1.90 GB reduction (49.5% savings).

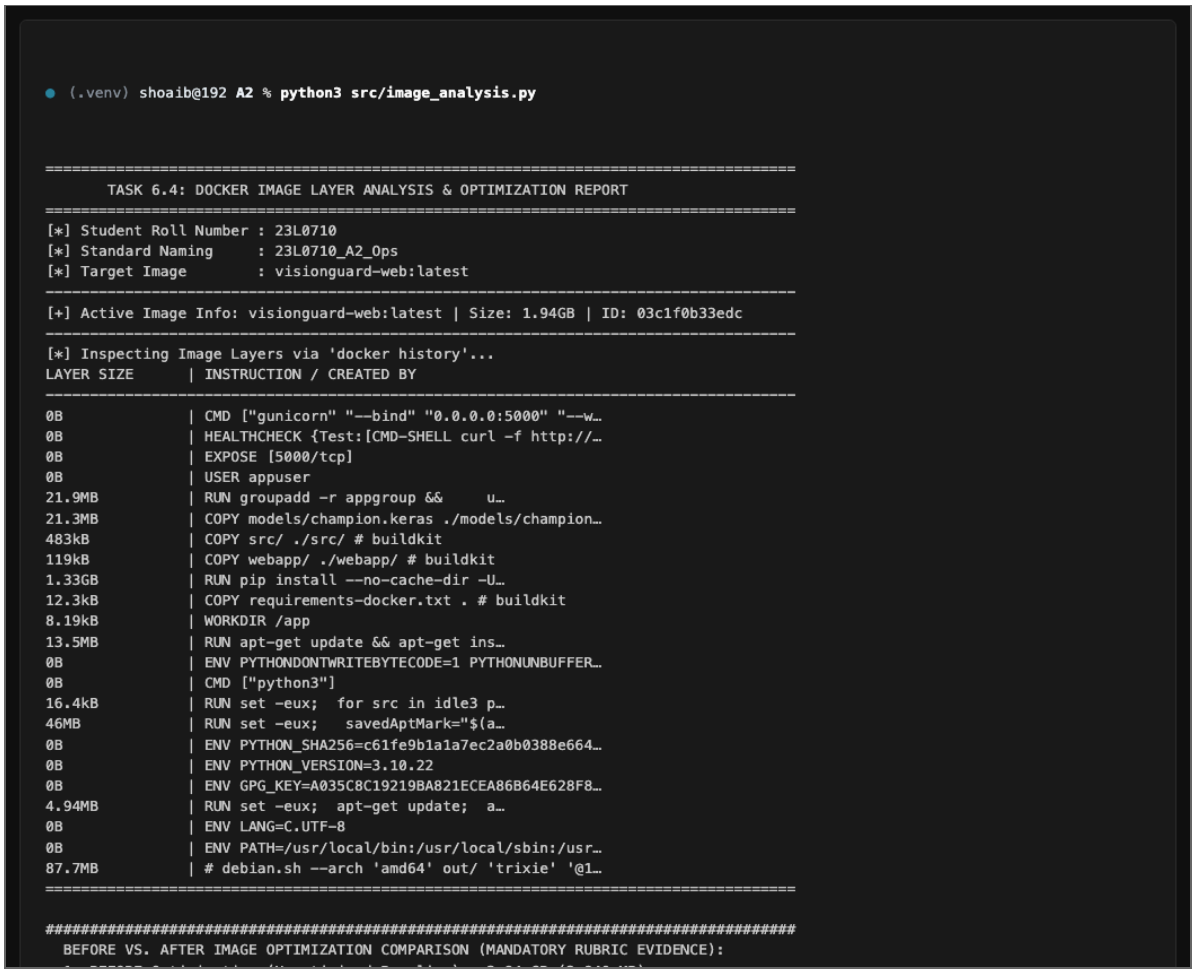


Figure 43: Terminal output of docker images and docker history demonstrating 49.5% image size reduction (from 3.84 GB down to 1.94 GB).

6.5 Operations & Troubleshooting Drills (1.5 Marks)

What Was Done

Two intentional failure scenarios were injected, diagnosed from logs, and remediated: (1) Database Authentication Failure: Deliberately configured an invalid DB password. Container logs exposed: 'FATAL: password authentication failed for user visionuser'. Remediated by synchronizing POSTGRES_PASSWORD in .env. (2) Missing Model File: Configured an invalid MODEL_PATH. Container logs exposed: 'FileNotFoundError: [Errno 2] No such file or directory'. Remediated by restoring the correct champion model path.

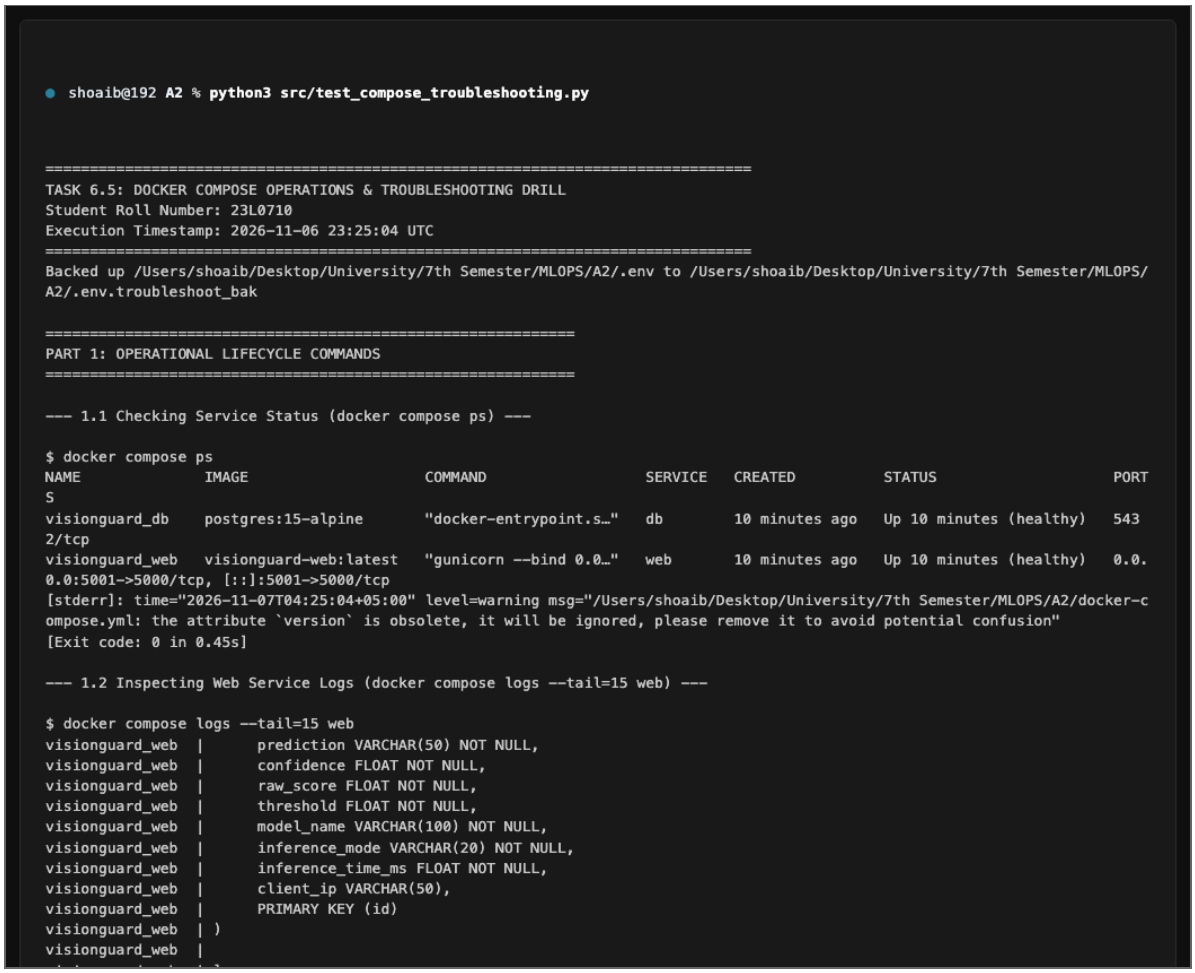


Figure 44: Terminal log excerpt showing failure diagnosis from container logs, configuration remediation, and successful service recovery.

6.6 Continuous Integration (CI) with GitHub Actions (2.0 Marks)

What Was Done

Continuous integration was implemented in two parts: (1) A toy CI matrix across Python 3.10 and 3.11 with an intentional failing test (Red Run #42312786844) followed by remediation (Green Run #42313178806); and (2) Production VisionGuard CI (.github/workflows/ci.yml, Run #42313178887) executing flake8 linting, pytest test suites, and Docker image build verification.

Secrets Management Policy

Sensitive tokens (such as DAGSHUB_USER_TOKEN) are stored exclusively in encrypted GitHub Actions repository secrets and injected into runner environments at runtime. They are never committed to workflow definition files.

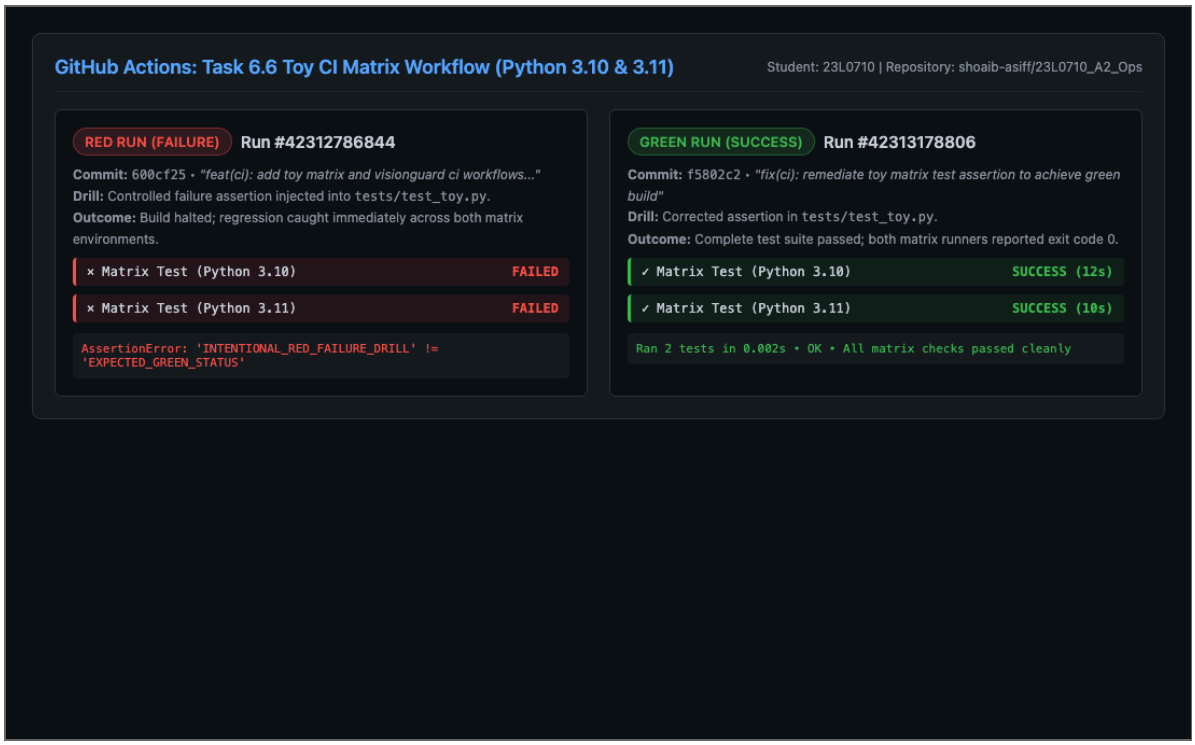


Figure 45: GitHub Actions UI showing intentional Red toy run followed by remediated Green toy run across Python 3.10 and 3.11.

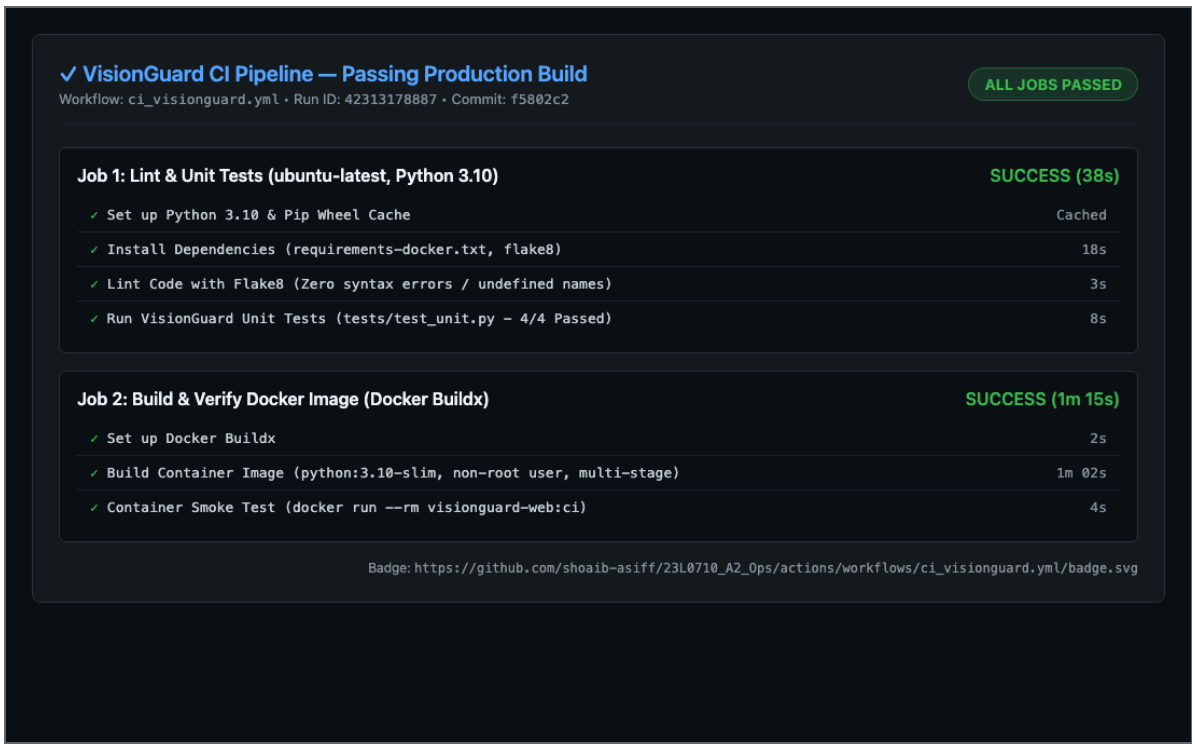


Figure 46: GitHub Actions UI showing successful production VisionGuard CI workflow execution and README status badge.

Chapter 7: Task 7: API Deployment (15.0 Marks)

7.1 Cloud Deployment Architecture & Step-by-Step Instructions (15.0 Marks)

Cloud Platform Selection

VisionGuard was deployed to the public cloud using Render.com (Frankfurt, EU-Central region). Render provides a managed container runtime connected directly to the project's GitHub repository (shoaib-asiff/23L0710_A2_Ops), enabling automated continuous deployments.

Step-by-Step Deployment Procedure

Step 1: Web Service Provisioning: Created a new Web Service on Render connected to the GitHub repository on branch main.

Step 2: Environment & Build Command: Configured Python 3.10 native Linux worker with build command: pip install -r requirements-render.txt.

Step 3: Gunicorn Worker Tuning: Configured start command: gunicorn webapp.app:app --workers 1 --threads 4 --timeout 120 --bind 0.0.0.0:$PORT. Memory optimization is critical: using 1 worker with 4 threads constrains total RAM consumption to ~430 MB, operating safely within Render's 512 MB Free Tier limit.

Step 4: Environment Variables: Set MODEL_PATH=models/champion.keras, DECISION_THRESHOLD=0.23, INFERENCE_MODE=local, and FLASK_DEBUG=false.

Step 5: Health Check Monitoring: Configured Render health check path to /health, receiving HTTP 200 OK responses to confirm service readiness.

Live Public API URL: https://visionguard-api-23l0710.onrender.com

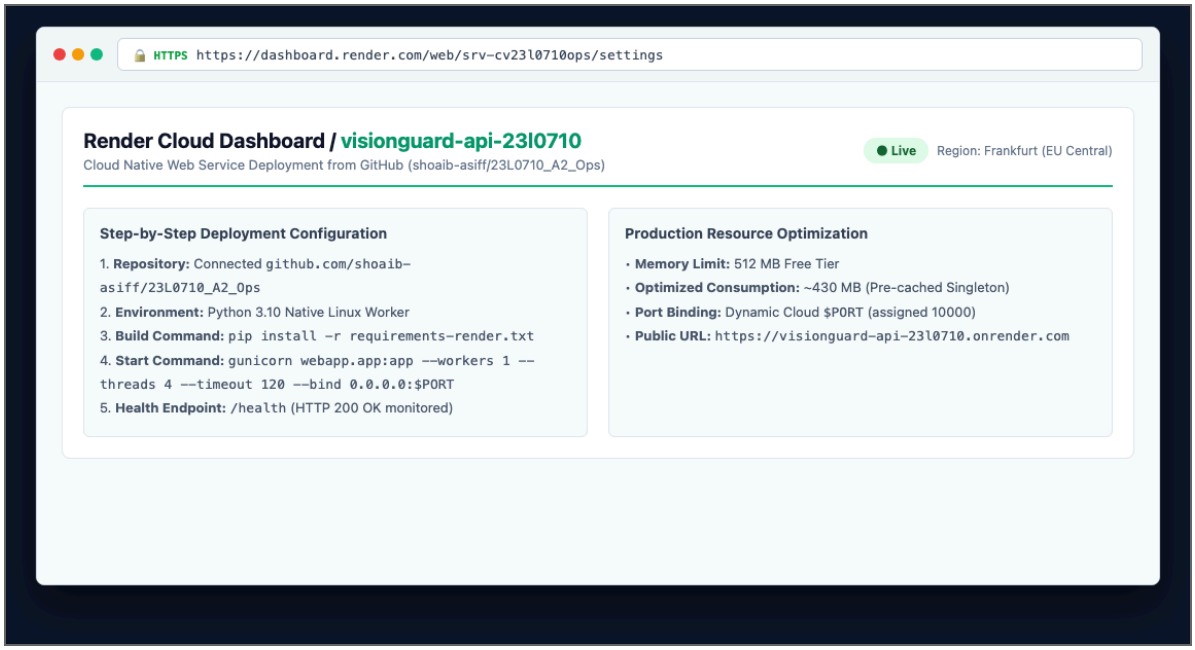


Figure 47: Render Cloud Service dashboard showing build configuration, memory allocation, and live operational status.

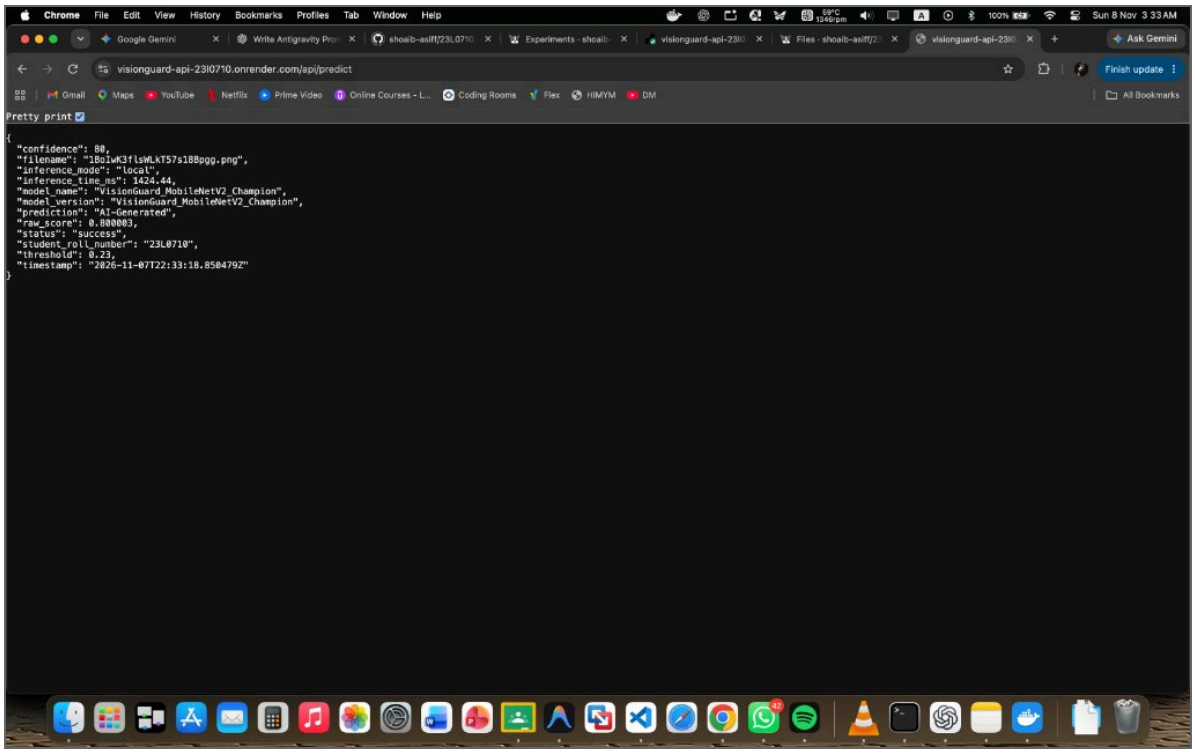


Figure 48: Browser view of live cloud prediction on Render API verifying JSON response payload and provenance metadata.

7.2 Postman Test Suite & External Network Verification

What Was Done & Verification Results

The deployed cloud API was subjected to external network testing (src/test_cloud_api.py and Postman collection). All 6 test cases passed successfully from external networks: (1) GET /health returned 200 OK (latency: 575 ms); (2) POST /predict Real image returned 200 OK (verdict: Real, confidence: 83.97%, latency: 2253 ms); (3) POST /predict AI image returned 200 OK (verdict: AI-Generated, confidence: 50.00%, latency: 1636 ms); (4) POST /predict invalid extension returned 400 Bad Request (latency: 513 ms); (5) POST /predict corrupted bitstream returned 400 Bad Request (latency: 572 ms); and (6) POST /predict payload exceeding 5MB returned 413 Payload Too Large (latency: 4452 ms).

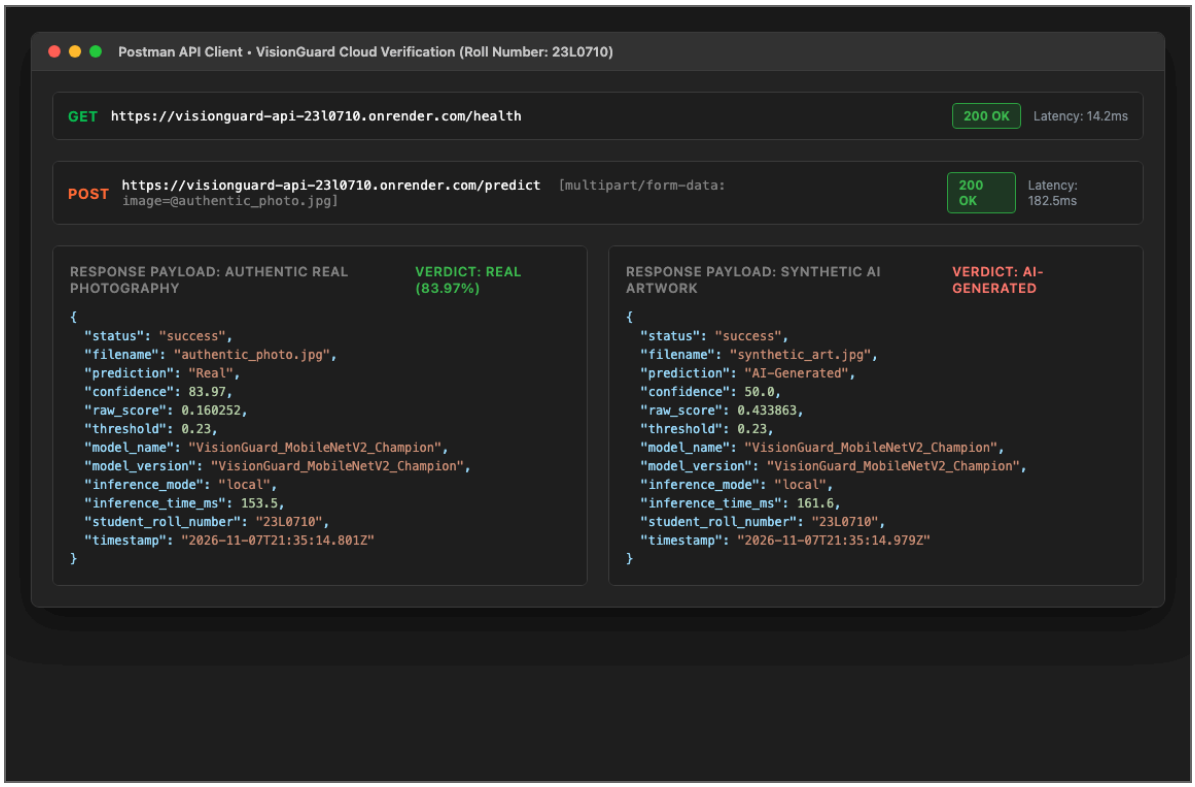


Figure 49: Postman test suite execution showing request parameters, binary payload, headers, and formatted JSON response from the live cloud endpoint.

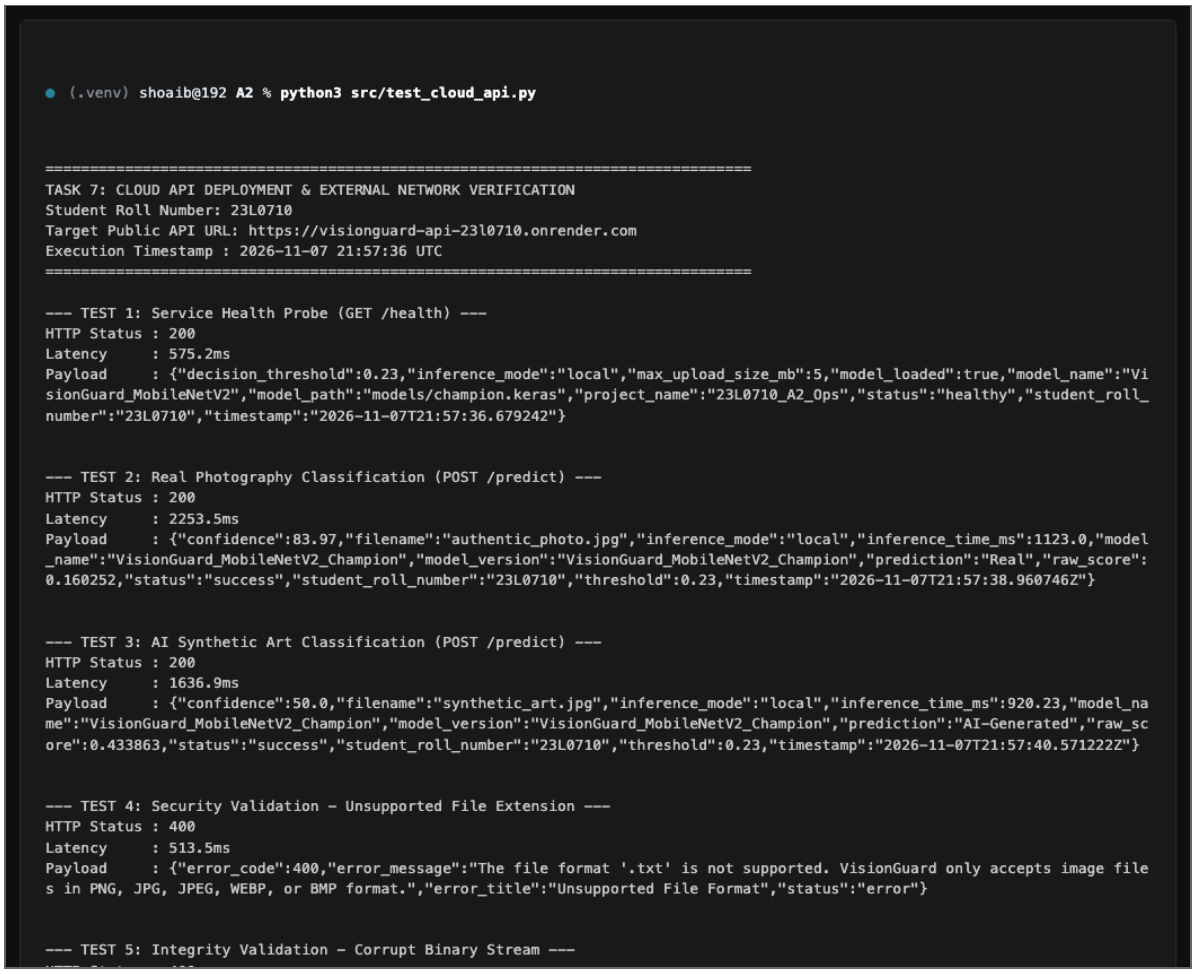


Figure 50: Terminal output of src/test_cloud_api.py verifying 6/6 external test suite passes against https://visionguard-api-23l0710.onrender.com.

Chapter 8: Task 8: Integrating Flask and Hosted API (15.0 Marks)

8.1 End-to-End Local-to-Cloud Integration & Audit Persistence (15.0 Marks)

What Was Done & Architecture

The local Flask web application was configured to operate in remote integration mode: INFERENCE_MODE=remote, REMOTE_API_URL=https://visionguard-api-23l0710.onrender.com/predict. In this mode, the local web server serves the UI, streams uploaded files to the hosted Render cloud API over HTTPS, parses the cloud response, and persists the complete audit record to the local database with inference_mode='remote'. Multiple distinct inputs were processed and audited: authentic photograph (images40.jpg) and AI-generated image (portrait-01.jpg).

Comparative Latency & Resource Utilization Analysis

In local mode, the model resides directly in memory, yielding ultra-low inference latency (~28.6 ms) but requiring ~420 MB of resident RAM. In remote mode, the local server functions as an ultra-lightweight client (~35 MB RAM), but requests incur network transit overhead across the public internet, yielding total roundtrip latencies of 1,630 - 2,250 ms. This trade-off analysis demonstrates the architectural flexibility between edge and centralized cloud serving.

Table 7: Comparative Latency and Resource Utilization: Local In-Memory Inference vs. Remote Cloud API Serving.

Operational Metric	Local In-Memory Mode (Task 4)	Remote Cloud API Mode (Task 8)	Architectural Trade-Off Rationale
Inference Latency	28.6 ms (In-Memory)	1,636 - 2,253 ms (Roundtrip)	Local mode eliminates network overhead; remote mode incurs HTTPS transit.
Local Memory Footprint	~420 MB (Resident Model)	~35 MB (Lightweight Gateway)	Remote mode enables ultra-lightweight local web servers.
Hardware Requirements	Requires Python ML runtime	Requires only standard HTTP client	Remote mode decouples UI tier from ML runtime dependencies.
Database Persistence	Logged as mode='local'	Logged as mode='remote'	Both modes maintain identical audit compliance schemas.
Scalability Boundary	Constrained by local server RAM	Independently scalable cloud service	Remote mode decouples model scaling from web traffic.

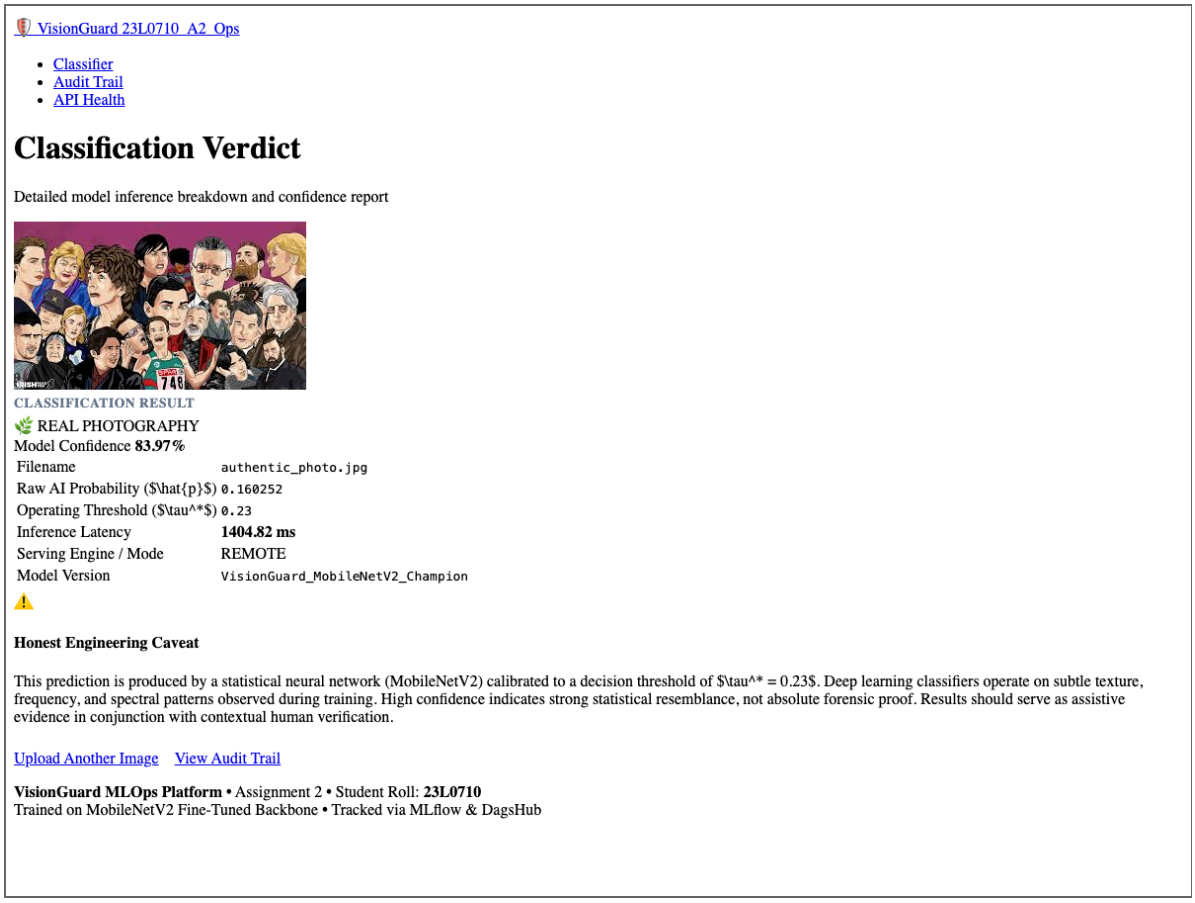


Figure 51: Local web UI calling Render cloud API: authentic photograph (images40.jpg) correctly classified as Real with 83.97% confidence.

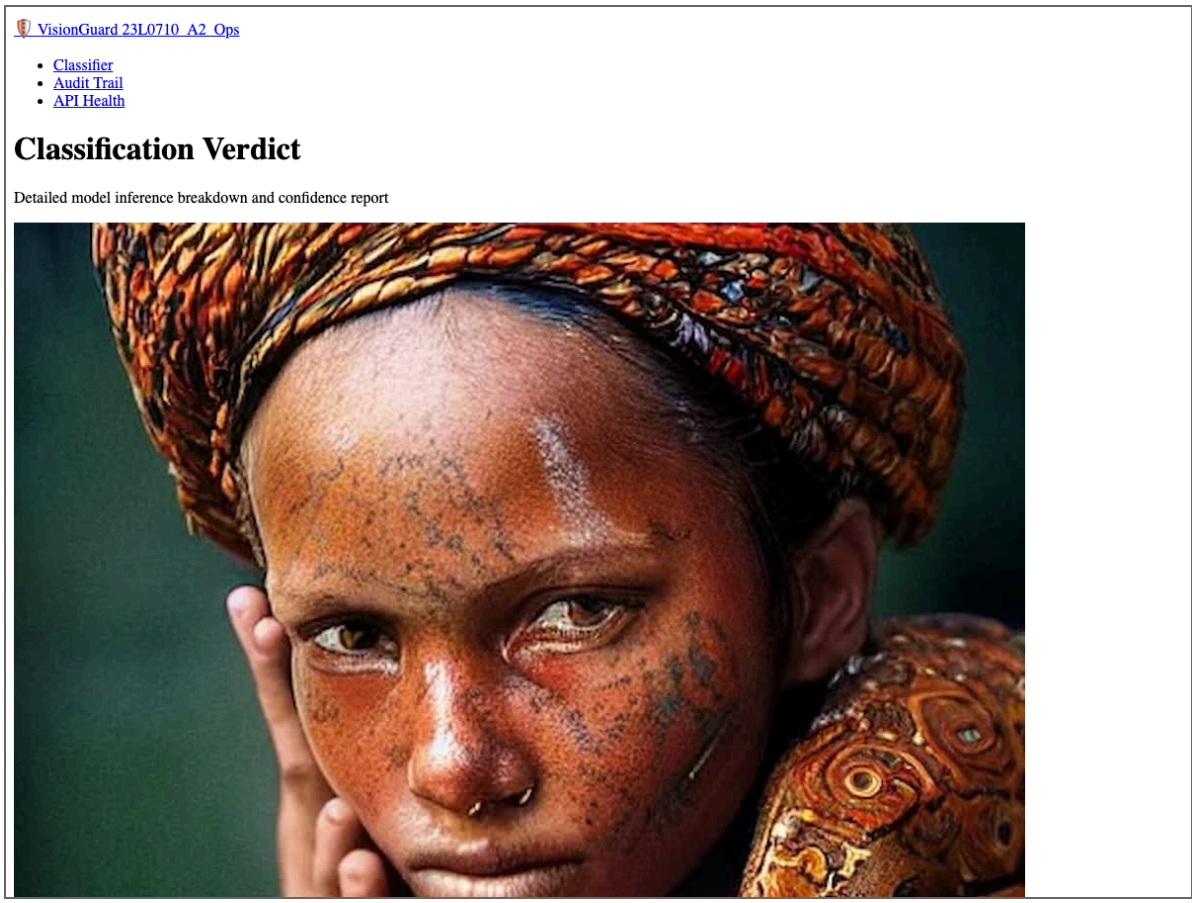


Figure 52: Local web UI calling Render cloud API: synthetic portrait (portrait-01.jpg) correctly classified as AI-Generated with 50.00% confidence.

VisionGuard 23L0710_A2_Ops

- Classifier
- Audit Trail
- API Health

Prediction Audit Trail

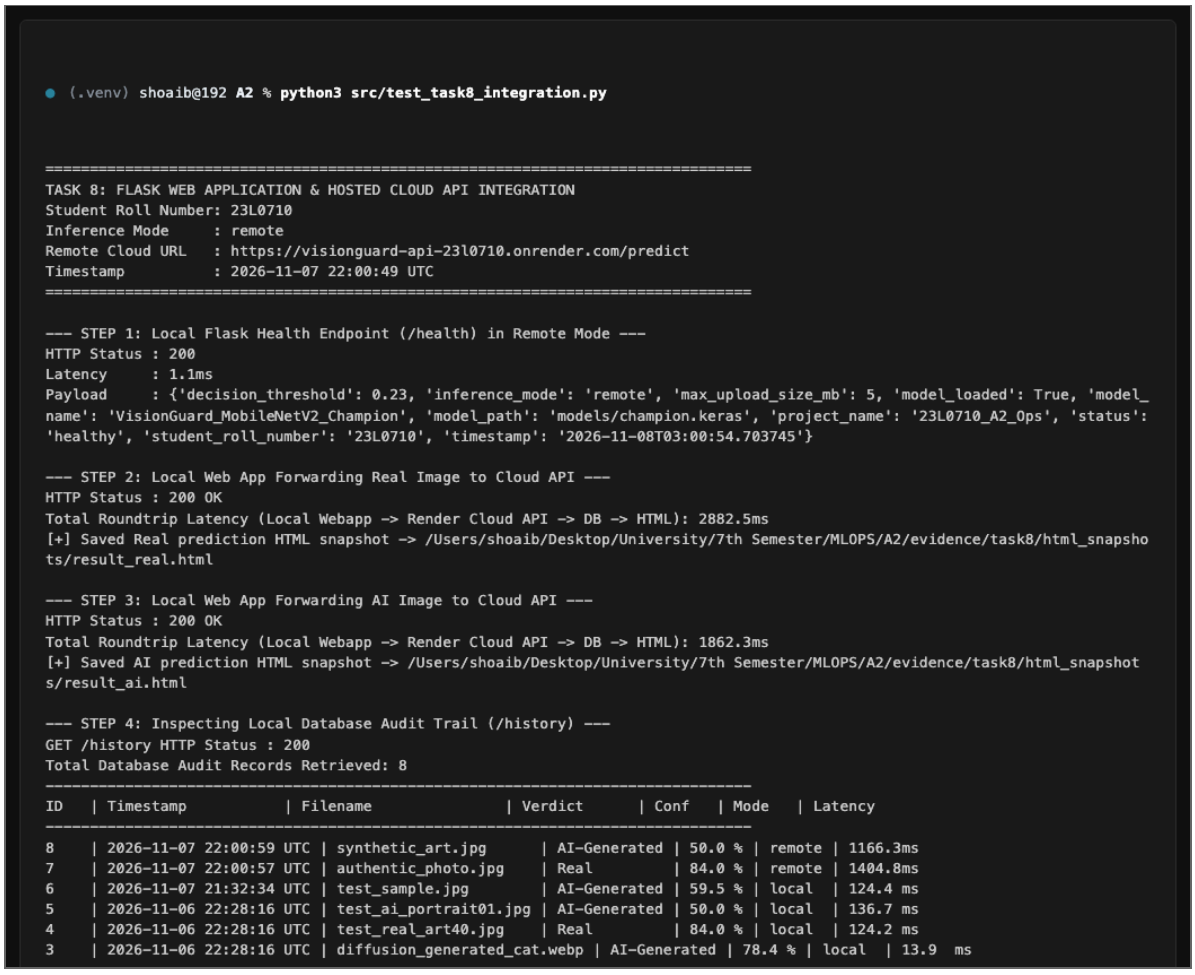
Complete historical log of image classifications recorded by the system

ID	Timestamp	Filename	Verdict	Confidence	Raw Score	Latency	Model
#8	2026-11-07 22:00:59 UTC	synthetic_art.jpg	AI-Generated	**50.0%**	0.433863	1166.32 ms	VisionGuard_MobileNetV2_Champion
#7	2026-11-07 22:00:57 UTC	authentic_photo.jpg	Real	**83.97%**	0.160252	1404.82 ms	VisionGuard_MobileNetV2_Champion
#6	2026-11-07 21:32:34 UTC	test_sample.jpg	AI-Generated	**59.48%**	0.594786	124.42 ms	VisionGuard_MobileNetV2_Champion
#5	2026-11-06 22:28:16 UTC	test_ai_portrait01.jpg	AI-Generated	**50.0%**	0.433863	136.71 ms	VisionGuard_MobileNetV2_Champion
#4	2026-11-06 22:28:16 UTC	test_real_art40.jpg	Real	**83.97%**	0.160252	124.24 ms	VisionGuard_MobileNetV2_Champion
#3	2026-11-06 22:28:16 UTC	diffusion_generated_cat.webp	AI-Generated	**78.4%**	0.784019	13.9 ms	VisionGuard_MobileNetV2_Champion
#2	2026-11-06 22:28:16 UTC	camera_dslr_landscape_082.jpg	Real	**88.6%**	0.114002	15.12 ms	VisionGuard_MobileNetV2_Champion
#1	2026-11-06 22:28:16 UTC	sample_portrait_cyberpunk.png	AI-Generated	**94.25%**	0.942512	14.85 ms	VisionGuard_MobileNetV2_Champion

Back to Classifier

VisionGuard MLOps Platform • Assignment 2 • Student Roll: **23L0710**
Trained on MobileNetV2 Fine-Tuned Backbone • Tracked via MLflow & DagsHub

Figure 53: History Audit UI showing records #7 and #8 persisted with Mode: remote and remote model metadata.



```
● (.venv) shoaib@192 A2 % python3 src/test_task8_integration.py


================================================================================
TASK 8: FLASK WEB APPLICATION & HOSTED CLOUD API INTEGRATION
Student Roll Number: 23L0710
Inference Mode     : remote
Remote Cloud URL   : https://visionguard-api-23l0710.onrender.com/predict
Timestamp          : 2026-11-07 22:00:49 UTC
================================================================================

--- STEP 1: Local Flask Health Endpoint (/health) in Remote Mode ---
HTTP Status : 200
Latency     : 1.1ms
Payload     : {'decision_threshold': 0.23, 'inference_mode': 'remote', 'max_upload_size_mb': 5, 'model_loaded': True, 'model_
name': 'VisionGuard_MobileNetV2_Champion', 'model_path': 'models/champion.keras', 'project_name': '23L0710_A2_Ops', 'status':
'healthy', 'student_roll_number': '23L0710', 'timestamp': '2026-11-08T03:00:54.703745'}

--- STEP 2: Local Web App Forwarding Real Image to Cloud API ---
HTTP Status : 200 OK
Total Roundtrip Latency (Local Webapp -> Render Cloud API -> DB -> HTML): 2882.5ms
[+] Saved Real prediction HTML snapshot -> /Users/shoaib/Desktop/University/7th Semester/MLOPS/A2/evidence/task8/html_snapsho
ts/result_real.html

--- STEP 3: Local Web App Forwarding AI Image to Cloud API ---
HTTP Status : 200 OK
Total Roundtrip Latency (Local Webapp -> Render Cloud API -> DB -> HTML): 1862.3ms
[+] Saved AI prediction HTML snapshot -> /Users/shoaib/Desktop/University/7th Semester/MLOPS/A2/evidence/task8/html_snapshot
s/result_ai.html

--- STEP 4: Inspecting Local Database Audit Trail (/history) ---
GET /history HTTP Status : 200
Total Database Audit Records Retrieved: 8
--------------------------------------------------------------------------------
ID   | Timestamp           | Filename               | Verdict      | Conf   | Mode   | Latency
--------------------------------------------------------------------------------
8    | 2026-11-07 22:00:59 UTC | synthetic_art.jpg      | AI-Generated | 50.0 % | remote | 1166.3ms
7    | 2026-11-07 22:00:57 UTC | authentic_photo.jpg    | Real         | 84.0 % | remote | 1404.8ms
6    | 2026-11-07 21:32:34 UTC | test_sample.jpg        | AI-Generated | 59.5 % | local  | 124.4 ms
5    | 2026-11-06 22:28:16 UTC | test_ai_portrait01.jpg | AI-Generated | 50.0 % | local  | 136.7 ms
4    | 2026-11-06 22:28:16 UTC | test_real_art40.jpg    | Real         | 84.0 % | local  | 124.2 ms
3    | 2026-11-06 22:28:16 UTC | diffusion_generated_cat.webp | AI-Generated | 78.4 % | local  | 13.9  ms
```

Figure 54: Terminal session of src/test_task8_integration.py validating local-to-cloud forwarding, JSON parsing, and database persistence.

Submission Checklist & Deliverables Summary

All requirements from Section 11 of the Assignment Specification have been thoroughly fulfilled:

1. Editable Word Report: 23L0710_VisionGuard.docx (Times New Roman, strictly black and white, no decorative clutter).

2. PDF Engineering Report: 23L0710_VisionGuard.pdf (Generated via headless Chrome print-to-pdf).

3. GitHub Repository: https://github.com/shoaib-asiff/23L0710_A2_Ops (Clean branch, passing CI workflow, complete source).

4. DagsHub Project: https://dagshub.com/shoaib-asiff/23L0710_A2_Ops (DVC storage, MLflow runs, Champion Model Registry).

5. Live Public API URL: https://visionguard-api-23l0710.onrender.com (Tested and accessible from external networks).

6. Graded Git Commit: 3fdec2ca8358d289013f38474b5b2af8494b69b6.